



3D Reconstruction from Multi-View Video Data with Advanced Machine Learning and Optimization Techniques

Master's Thesis

submitted by
Mathieu Hobbah

Matricule Number: 500935

Modeling of Cognitive Processes
Computer Vision and Remote Sensing
Fakultät IV

September 2025

1. Reviewer : Prof. Dr.-Ing. Olaf Hellwich
2. Reviewer : Prof. Dr. Guillermo Gallego

Declaration of Authorship

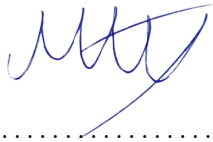
I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the *Principles for Ensuring Good Research Practice at TU Berlin* dated 15 February 2023.

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, 26.09.2025



.....
Signature

Abstract

Tracking mouse body parts in video is often incomplete due to occlusions and motion blur such that - e.g. - subsequent action and behavior analysis is impeded, particularly when working with only one camera. The conceptual work *Tracking Mouse from Incomplete Body-Part Observations and Deep-Learned Deformable-Mouse Model Motion-Track Constraint for Behavior Analysis* [1] introduces a whole pipeline for the complete track reconstruction of a mouse's body parts from multi-view video data. Firstly, the mouse is modeled as a 3D skeleton where body part positions are estimated by 3D triangulation and least-squares bundle adjustment with a global motion-track smoothness constraint. Nonetheless, these methods are not capable of recovering body parts when they are occluded in all camera views. This is why the pipeline needs an additional method to infer missing body part positions from the visible skeleton and Deep Learning (DL) seems particularly suited for this kind of task. This thesis focuses on the implementation of such a DL method by analyzing and comparing different networks on a simulated dataset with artificial occlusions. To make an informed network selection, an extensive literature review is conducted beforehand on various methods that could fit in the pipeline. Finally, our networks overall achieve skeleton reconstruction across various conditions but with some contrasted results.

Kurzfassung

Das Tracking von Maus-Körperteilen in Videos ist aufgrund von Verdeckungen und Bewegungsunschärfe häufig unvollständig, sodass beispielsweise die anschließende Analyse von Aktionen und Verhaltensweisen erschwert wird – insbesondere, wenn nur eine Kamera verwendet wird. Die konzeptionelle Arbeit *Tracking Mouse from Incomplete Body-Part Observations and Deep-Learned Deformable-Mouse Model Motion-Track Constraint for Behavior Analysis*[1] stellt eine vollständige Pipeline zur Rekonstruktion der Mauskörperteile aus Multi-View-Videodaten vor. Zunächst wird die Maus als 3D-Skelett modelliert, wobei die Positionen der Körperteile mittels 3D-Triangulation und Least-Squares-Bundle-Adjustment unter einem globalen Glättungs- bzw. Bewegungspfad-Constraint geschätzt werden. Diese Methoden können jedoch keine Körperteile rekonstruieren, die in allen Kameraperspektiven verdeckt sind. Daher benötigt die Pipeline eine zusätzliche Komponente, um fehlende Körperteilpositionen aus dem sichtbaren Skelett abzuleiten – und Deep Learning (DL) erscheint für diese Aufgabe besonders geeignet. Diese Arbeit konzentriert sich auf die Implementierung einer solchen DL-Methode, indem verschiedene Netzwerke auf einem simulierten Datensatz mit künstlichen Verdeckungen analysiert und verglichen werden. Um eine fundierte Modellauswahl zu treffen, wird zuvor eine umfassende Literaturrecherche zu geeigneten Verfahren durchgeführt. Abschließend erreichen unsere Netzwerke insgesamt eine Skelettreakonstruktion unter unterschiedlichen Bedingungen, wenn auch mit teils kontrastreichen Ergebnissen.

Contents

List of Figures	vii
List of Tables	ix
1. Introduction	1
2. Literature review	5
2.1. LSTM-based Approaches	6
2.1.1. Overview of LSTM Architecture	6
2.1.2. Representative Methods	7
2.2. GCN-based approaches	11
2.2.1. Overview of ST-GCN Architecture	11
2.2.2. Representative Methods	13
2.3. Transformer-based methods	18
2.3.1. Overview of Transformer Architecture	18
2.3.2. Representative Methods	20
3. Methodology	25
3.1. Dataset	25
3.1.1. Data Generation [1]	25
3.1.2. Statistics	26
3.1.3. Masking	27
3.2. Exploration	30
3.2.1. Metrics	31
3.2.2. Networks	34
4. Evaluation	39
4.1. Test Sets	39
4.2. Baseline	39
4.3. Results	41
4.3.1. Coordinates	41
4.3.2. Parts	43
4.3.3. Frames	48
5. Conclusion	51
A. Annex	53
A.1. Training	53
A.2. LSTM Exploration	53
A.3. BRA	54
A.4. Data Augmentation	55
Bibliography	57

List of Figures

1.1. Lockbox experiment [1]	2
2.1. LSTM architecture [28]	6
2.2. BRA architecture [7]	7
2.3. BRA performance in reconstruction [7]	8
2.4. A-LSTM+LS framework [8]	9
2.5. A-LSTM architecture [8]	9
2.6. Spatio-temporal graph [38]	12
2.7. Partitioning strategies [38]	12
2.8. ST-GCN framework [38]	13
2.9. Partition strategy [6]	13
2.10. Network architecture [6]	14
2.11. Quantitative results [6]	14
2.12. Qualitative results [6]	14
2.13. Adaptative Graph Convolutional Layer (AGCN) [42]	16
2.14. 2s-AGCN framework [42]	16
2.15. MS-G3D partition strategy [43]	17
2.16. Res-GCN framework [44]	17
2.17. Transformer architecture [46]	18
2.18. SkeltonMAE masking strategy [3]	20
2.19. Typical GCN vs STTFormer self-attention schemes [48]	21
2.20. STTFormer data preprocessing [48]	21
2.21. SkeletonMAE framework [3]	22
2.22. SkeletonMAE skeleton reconstruction [3]	23
3.1. Mouse model	25
3.2. Mean bone lengths with standard deviation for one sequence of 100 time steps . .	26
3.3. Coordinates masking	27
3.4. Parts masking	27
3.5. Frames masking	28
3.6. Example of a result curve	30
3.7. Embedding Dimensions	34
3.8. Dataset parameters	35
3.9. Regularization	36
3.10. ST-GCN with the full architecture	37
3.11. Symmetry Strategy for the left ear	38
3.12. ST-GCN	38
4.1. θ is the angle span [53]	39
4.2. Baseline metrics across the test sets for the Coordinates masking	39
4.3. Baseline metrics across the test sets for the other maskings	40
4.4. PCK for Frames masking	40
4.5. MPE	41
4.6. Other metrics	42
4.7. MPE	43

4.8. MPE per body part on Seed for a missing rate of 0.8	43
4.9. Other metrics	44
4.10. Example of a skeleton reconstruction for 4 occluded parts on Seed. Z-diff is the mean difference between predicted and ground-truth Z coordinates.	45
4.11. Example of a skeleton reconstruction for 4 occluded parts on Angle	46
4.12. Tracking of reconstructed body parts for AGCN on Seed	47
4.13. Tracking of reconstructed body parts for LSTM on seed	47
4.14. MPE	48
4.15. Other metrics	48
4.16. Example of skeleton reconstructions during the occlusion of 10 frames on Speed .	49
A.1. Typical training curves	53
A.2. MPE	53
A.3. BLE	54
A.4. BRA [7]	54
A.5. Data augmentation for Parts tested on Angle	55

List of Tables

2.1. Recognition accuracy (%) under different occlusion scenarios [36]	15
3.1. Biologically realistic rigid mouse model in mouse's coordinate system	26
3.2. Example of MPE values (mm)	31
3.3. Example of PCK (%)	32
3.4. Example of BLE values (mm)	33

1. Introduction

Accurate analysis requires accurate observations. This is the challenge faced in animal behavior classification, in particular with mice. In a previous work [2] that aimed to investigate how mice solve problems in a lockbox experiment, the observers used three cameras to reduce the impact of obstructions on the pose tracking. However, they still had difficulties to correctly assess the behaviors of the mice because the use of multiple cameras does not guarantee a complete observation of their bodies at any time. In such an experiment, mice movements and interactions are very complex. To understand how a mouse gets to a solution, detailed information on the sequences of the mouse’s movements are needed, as well as its interactions with the environment and with the other mice. Therefore, the conceptual work [1] proposes a pipeline to tackle the occlusion and noise problem in the skeletal data. The computational pipeline outputs 3D positions from the 2D images of the cameras (3D pose estimation). The first step is a spatial resection that computes extrinsic camera parameters (orientation and translation) in a projection matrix from fixed and observed elements in the lockbox. This gives a transformation from the 3D global coordinates to the 3D camera coordinate system. Another spatial resection is computed but with the mouse’s body parts in place of the fixed elements, so there is a projection matrix for any given t . Then, a least-squares bundle adjustment optimizes these projection matrices together with the 3D positions of the body parts. In order to ensure the smoothness of the positions over time, a motion-track constraint is applied on the projection matrices of the mouse’s body parts. This constraint consists in comparing the estimated projection matrix at a given time t with a cubic spline interpolation of projection matrices at time $t - 2$, $t - 1$, $t + 1$ and $t + 2$. Besides, the mouse is assumed to be “rigid” and the deformable (i.e, real) body parts positions are treated as noise on the image coordinates in a first approximation. A Deep Learning (DL) neural network is trained to recover real body part positions from the body part positions of the rigid mouse model. This thesis focuses on the implementation of the DL method.

DL’s large variety of network makes it applicable to many different task and it is widely used for skeletal data reconstruction. DL is a very powerful tool for learning underlying dependencies of the data, which are two-fold in skeletal motion data (spatial and temporal). Very modern networks leverage the Transformer architecture [3] [4] and Graph Convolutional Networks (GCN) [5] [6], but earlier methods were primarily LSTM-based [7] [8] [9]. There is no well-established state of the art for skeletal data reconstruction, as it is not typically treated as an independent task but rather as a component within broader problems such as pose estimation and action recognition. Furthermore, skeletal data (joints, edges, angles, etc.) and reconstruction objectives (i.e, distance, pose, etc.) can substantially varies from one situation to another which makes the development of global methods and therefore the establishment of a general benchmark very challenging. Instead, performance of these methods is most of the time evaluated on the final end task. The problem can also be approached as an imputation task, treating the skeletal data as a multivariate time series with missing values. There exists a state of the art in this area and the networks are regarded as forecasting models. Simple LSTM can be classified with these methods, but state-of-the-art methods leverage way more complex architectures such as diffusion models [10] or Transformers [11] [12]. Except for the LSTM, we do not consider such imputation methods in our thesis and rather focuses on methods tailored to handle skeletal data. These methods are plenty in the field of Human Action Recognition (HAR). DL, or more generally Machine Learning (ML), is also used in animal behavior classification but it is not as ubiquitous as it is in HAR [13]. In order to achieve high performance, DL networks need large

1. Introduction

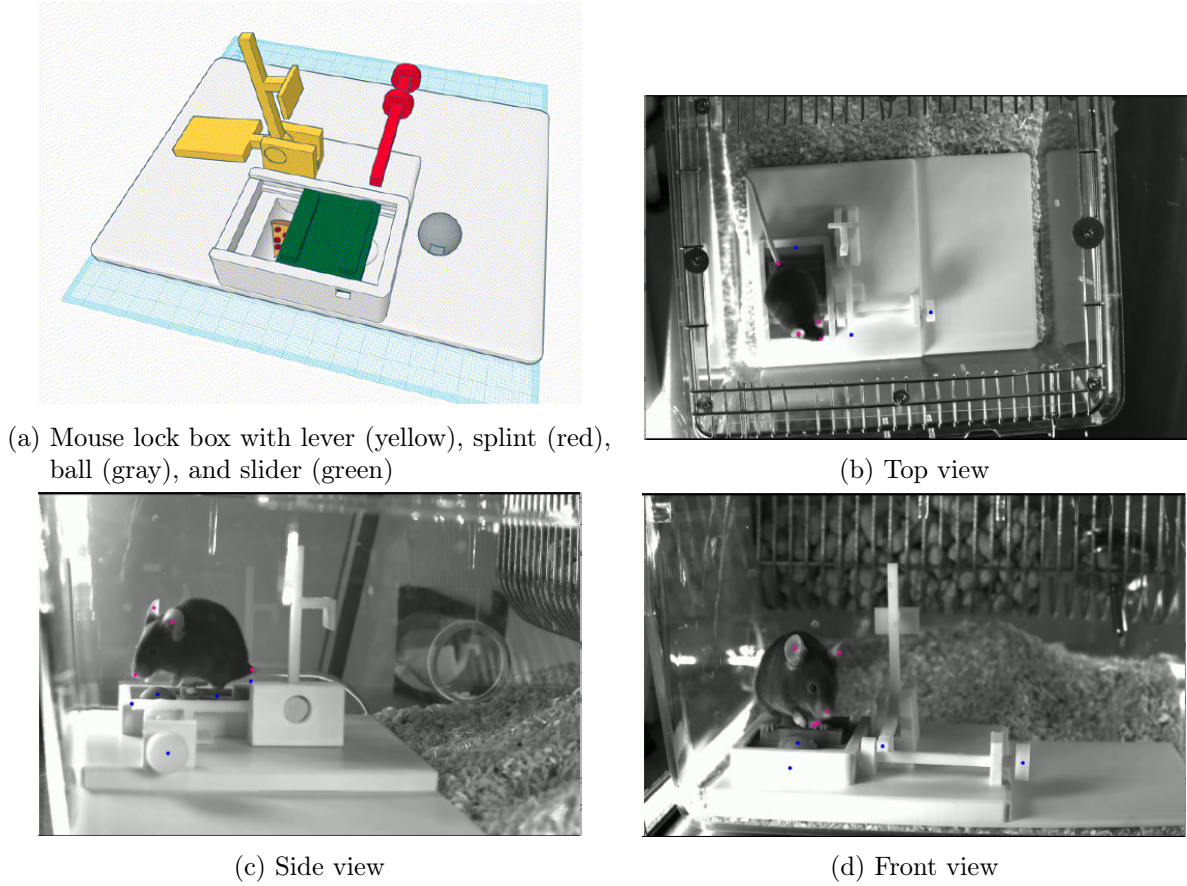


Figure 1.1.: Lockbox experiment [1]

and high-quality datasets and many such datasets are publicly available for HAR [14][15][16][17]. Conversely, animal datasets are rarer because animal behavior research has traditionally relied on expert-driven, frame-by-frame video annotation; a process that is expensive, labor-intensive, and difficult to crowdsource due to the expertise required for accurate behavior identification [18]. Nonetheless, recent years have seen a surge in the number of large-scale and accessible mouse datasets, thereby expanding DL in the particular mouse behavior recognition field and enabling the development of fully automated methods. For instance, the Mouse Action Recognition System (MARS) [19] proposes an exhaustive automated pipeline that classifies mice behavior from a video of two interacting mice, and introduces three datasets with a total of 28 hours of video for the training and testing of the different stages of the pipeline. Mars also includes a pose estimation system that is robust to occlusions and motion blur. It detects the two mice on 2D images using the Inception ResNet v2 [20] and leverages a stacked hourglass CNN [21] to perform the skeletal pose estimation. Finally, the behavior classification is carried out by ML networks. MARS achieves human-level performance on the classification tasks without introducing additional elements to explicitly handle the occlusions and just relying on the robustness of the stacked hourglass architecture. However, MARS considers “simple” actions such as sniff face, sniff body etc. while we want to characterize a succession of more complex and brief actions (Figure 1.1 *e.g.* . *first lever, then splint, then ball, finally slider lid, which is the only sequence of mechanisms that allows access to the hidden reward, an oats flake* from [1]).

With this aim, the Mouse Lockbox Dataset [22] was created with more than 110 hours of playtime from three cameras and displaying images of mice solving lockbox problems. Around 13% of the dataset is labelled with 20 frame-level human annotations. By using DeepLabCut [23] for the pose estimation, we get image coordinates of the mouse body parts observed. This enables the

application of the least-squares bundle adjustment and the rest of the pipeline subsequently. Nevertheless, we do not work with this dataset because it does not provide ground-truth skeletal data, but only videos. For this reason, we work with a simulated dataset so that we can challenge our networks across various conditions that we implement. A first type of masking occlude coordinates randomly and is useful for quantitative comparison. A second type of masking randomly occludes body parts for a given time interval and the last masking occludes successions of frames starting from random time steps. This dataset is also of limited size and models the motion of only one mouse, which enables the training and test of many configurations. We analyze and compare a simple LSTM and a GCN methods that will be thoroughly described in the following chapters. We do not implement any Transformer solution, as these architectures often require very large datasets and long periods of training. Finally, our networks can reconstruct the skeletons when the occlusion conditions are not too severe, with the LSTM achieving the best performance.

2. Literature review

As mentioned in the introduction, reconstructing skeletal motion data is rarely the end goal in of a method. Instead, it serves as an intermediate stage aimed at enhancing the downstream task that represents the primary problem to be addressed. HAR is a branch of computational science where methods involving skeletal motion data and DL are widespread. It became really popular because of its wide range of application across different fields such as healthcare, human-computer interaction, or even sociology [24]. Consequently, researchers developed many techniques to classify human activities, from statistical methods to very deep learning architectures.

Thus, this literature review will present some HAR methods and we decide to introduce the typical datasets beforehand. The rise of the DL networks in HAR was supported by very large and freely accessible datasets, notably the NTU-RGB+D 60 [14] dataset that comprises 56, 880 RGB video samples captured from 80 views of 40 different human subjects enacting 60 action classes. In addition to the RGB images, NTU-RGB+D 60 also provides 3D skeleton data. A new version of NTU-RGB+D 60, NTU-RGB+D 120 [15], extends the action classes to 120, the number of subjects to 106 and the number of views to 155. Kinetics [16] and Human3.6M [17] are two others notable datasets. Kinetics only provides RGB videos but representing 400 human action classes over 306, 425 clips. Regarding Human3.6M, this dataset is remarkable in size, with more than 3.6 million human poses available in both RGB and skeleton modalities.

The literature review will be conducted in three sections, each one corresponding to a classical architecture. Firstly, we will tackle the Long-Short Term Memory (LSTM)-based approaches. In a second part will come the solutions leveraging GCN and in a last part the ones using Transformer architectures. Some of the methods presented below will be applied as such or adapted before being applied to our specific case.

2.1. LSTM-based Approaches

The LSTM architecture was designed to solve the problem of the exploding and vanishing gradients during the Back-Propagation Through Time (BPTT) in the classical Recurrent Neural Networks (RNNs). It has been one of the most popular DL network architecture to address time-related problems because of its ability to handle sequential data. Its large range of applications, from basic time series analysis to speech recognition, confirm the high reliability of the architecture. Despite their widespread use in the ML community, we revisit LSTM here to provide necessary background.

2.1.1. Overview of LSTM Architecture

LSTM [25] solve the classical RNN problems by adding special units called memory blocks in the recurrent hidden layer. The original LSTM cell is an element of a memory block and there is one cell for every hidden dimension. However, the “memory block” term is little used in the modern terminology and “cell” more or less refers to the original memory block now. We will use the latter in the following of this thesis. The particularity of the LSTM is to manage temporal features in the units we call cell, enabling the capture of long-term dependencies. The “gates” update the cell state and there are three different kinds: the input gate, the forget gate and the output gate. The input and output gates respectively modulate what gets into and out of the cell while the forget gate enables the tracking of the cell state [26]. All the elements of an LSTM cell are computed as follows [27]:

$$\begin{aligned}
 i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
 f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
 g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\
 o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned}$$

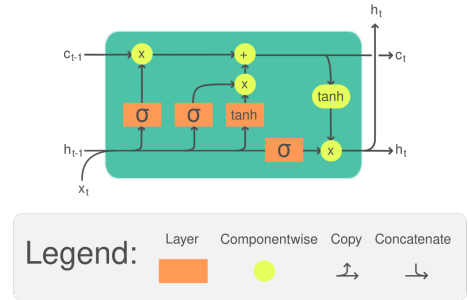


Figure 2.1.: LSTM architecture [28]

where i_t , f_t , and o_t are, respectively, the input, forget and output gates. g_t is the candidate state, sometimes called the cell gate. h_t and c_t are the hidden and cell states. All elements cited previously are defined at time t . W and b are the weight and bias matrices. σ denotes the sigmoid function while $\odot$ denotes the Hadamard product (element-wise product).

The LSTM proved to be really efficient in speech recognition, significantly improving machine translations on Google Translate and the answers of Amazon’s Alexa. Nonetheless, Natural Language Processing (NLP) is not its only area of application in ML. Van Houdt et al.[29] points that out in their survey on LSTM when they present its relevant applications. In addition to NLP, they give examples of applications for time series prediction, image and video captioning and more widely computer vision. While conventional LSTM architectures (adding layers, Bidirectional LSTM (BiLSTM), ...) are in most cases sufficient to solve time series prediction tasks, NLP and computer vision tasks require more complex frameworks where the LSTM is associated with other deep learning networks such as Convolutional Neural Networks (CNN). Moreover, the problems involving skeletal data in the survey leverage a hybrid CNN-LSTM architecture, which makes us think that the LSTM architecture as itself might not be enough to efficiently solve our task.

2.1.2. Representative Methods

BRA

Input reconstruction is a common unsupervised learning task and autoencoders [30] are widely employed to perform it. An autoencoder is more a framework than a network type strictly speaking. Autoencoders rely on an encoder to give a representation of the input data and a decoder to output the original input given the representation, but the encoder and decoder may be implemented using any type of DL architecture. The network is trained with its original input as the objective. Given the autoencoders ability to reconstruct inputs and the LSTM aptitude to handle sequential data, we decide to review an LSTM autoencoder (i.e, an LSTM encoder associated to an LSTM decoder). More precisely, the autoencoder we present is in fact a Denoising Autoencoder [31], since its original input is corrupted.

This network is the Bidirectional Recurrent Autoencoder (BRA) [7]. This architecture is specifically designed to address the challenges posed by noise and missing information in 3D skeletal motion data. Raw 3D skeletal data from motion capture devices like Microsoft Kinect often suffer from noise or incompleteness due to factors like calibration errors, sensor noise, poor resolution, and occlusions. The authors argues the utilization of DL for the skeletal motion data refinement. Non-data-driven methods such as Kalman filtering [32] or singular value thresholding [33] are fast but often fail to preserve human body structural information or spatial-temporal patterns, leading to unnatural results or inability to fix badly corrupted data. Data-driven methods, particularly those using DL networks, learn dependencies directly from the data during optimization, thereby preserving the structural information. .

Li et al. use BiLSTM to allow the autoencoder to infer information flowing from past to future and from future to past. Before being fed to the BiLSTM, the skeletal data is augmented through fully connected layers to exploit spatial relationships. The BRA follows a classic autoencoder design with a symmetrical encoder-decoder structure (Figure 2.2), trained using pairs of noisy and clean motion data.

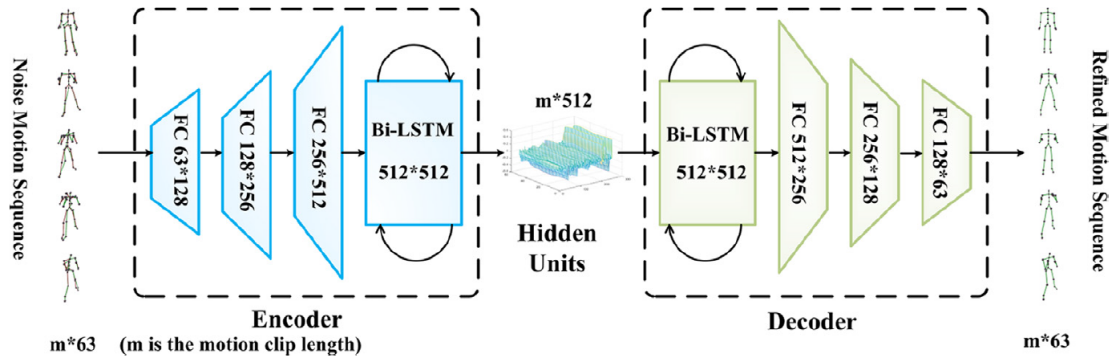


Figure 2.2.: BRA architecture [7]

2. Literature review

Another particularity of the method is the inclusion of smoothness and bone length constraints in the objective loss. The smoothness constraint prevents sudden variations in joint accelerations. The network is trained by minimizing this combined loss function $Loss = \lambda_1 Loss_p + \lambda_2 Loss_s + \lambda_3 Loss_b$ where $\lambda_1, \lambda_2, \lambda_3$ are the weight coefficients and:

$$Loss_p = \frac{1}{m \times n} \|Y - X\|_2 \quad Loss_s = \frac{1}{J} \|OY'\|_2 \quad Loss_b = \frac{1}{m \times (J-1)} \sum_{i=1}^m \sum_{b=1}^{J-1} \left| \|p_b^{i,1} - p_b^{i,2}\|_2 - l_b \right|$$

with

$$O = \begin{pmatrix} -1 & 1 & & & & 0 \\ 1 & -2 & 1 & & & \\ & 1 & \ddots & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & 1 & -2 & 1 \\ 0 & & & & 1 & -1 \end{pmatrix}_{(m+1) \times (m+1)}$$

m is the sequence length, J is the number of joints and $n = J \times 3$ is the number of individual coordinates at any given time. X is the complete input sequence to be reconstructed, Y is the output of the decoder while Y' repeats border elements of Y . The set $\{l_b \mid 1 \leq b \leq J-1\}$ is the bone-length sequence with b as the bone index, $p_b^{i,1}$ and $p_b^{i,2}$ are the 3D positions of the two end joints of the b th bone in frame i . $\|\cdot\|_2$ is the Euclidean distance.

BRA training strategy is also somewhat unusual, as the different component weights are adjusted in stages. Initially, positional precision is prioritized, then smoothness and bone length constraints are gradually given more weight, ensuring they do not significantly increase position loss. Extensive experiments were conducted by corrupting the CMU motion capture database [34] in three ways: first experiment adds Gaussian noise to the whole data, a second one removes 40% of the data and Gaussian noise is added to the reversed data, and a last one keeps five joints only and add Gaussian noise to their positions.

BRA is compared to previous similar architectures and a CNN. BRA consistently outperforms these methods across various noise types and evaluation metrics that are the position error, the bone-length error, and the jitter level (error relative to the smoothness constraint). Furthermore, BRA is non-action specific and it also handles different types of noise including Gaussian noise, randomly missing data, and sparse joint data. The model can be trained on data with high-amplitude noise and still perform on data with low amplitude noise even without any prior knowledge on the noise characteristics.

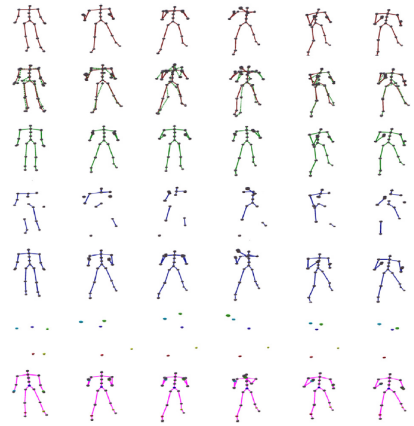


Figure 2.3.: BRA performance in reconstruction [7]

A-LSTM+LS

Ji et al. [8] propose a totally different approach to solve the missing data problem in human motion capture. They decided to opt for an attention mechanism on top of an LSTM architecture to capture very long-term dependencies. When introducing LSTM, we emphasized on its ability to retain information but it still leads to the loss of key information in contrast to the attention mechanism that is independent of the sequence length. The overall framework of the method involves two main stages. Firstly, an attention-based LSTM (A-LSTM) network is employed to estimate the positions of the missing markers. This is where all the temporal features are learned. After the initial estimation by the A-LSTM, a space affine transformation based on least-squares fitting serves as a spatial constraint. This step refines the estimated positions, significantly reducing the restoration error.

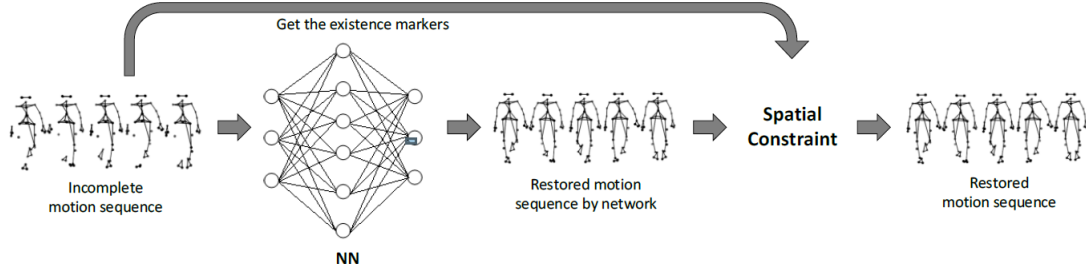


Figure 2.4.: A-LSTM+LS framework [8]

The network’s estimation capability primarily stems from the attention mechanism. A standard LSTM lacks processes all inputs uniformly, which can result in the loss of critical information when modeling long-term dependencies or when memory capacity is limited. By focusing on relevant parts of the input sequence, attention significantly improves both the effectiveness and efficiency of the network. Basically, the attention mechanism enables the network to better learn the temporal continuity.

The attention mechanism is added at the bottom layer of the LSTM. When processing the input for a given time step t , it takes into account the output of previous steps (up to the defined “attention length”). Attention is effectively computed in this manner:

$$a_i^t = \text{softmax} (v^T \tanh (W_\alpha c_t + W_\beta h_i))$$

$$\text{with } h_t = \sum_{i=t-n}^{t-1} a_i^t h_i$$

where W_α and W_β are the weights learned by the attention module. v is also a learned parameter and a_i^t denotes the attention weight that gets the correlation between step i and step t . c_t and h_t are the cell and hidden states at step t .

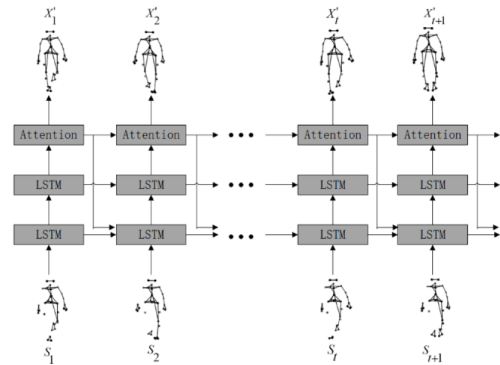


Figure 2.5.: A-LSTM architecture [8]

In this way, the process allows the network to “attend” to relevant past information. Ji et al. design two kinds of experiments to demonstrate their results. The first one fixes the number of missing markers and makes the missing time gap vary. The second one fixes the missing time gap but makes the number of missing markers vary. Both experiments clearly demonstrate the benefits of the attention mechanism. Indeed, the RMSE

2. Literature review

of A-LSTM is smaller than the RMSE of plain LSTM across different motion types (basketball, walk, boxing). This directly proves that the attention mechanism introduced is effective and helpful in the problem of missing data recovery. The integration of attention mechanism contributes to the robustness of the method to the length of missing gaps and the number of missing markers. The A-LSTM method showed more stability and less increase in error compared to traditional methods like the method from Burke et al. that uses Kalman filters [35].

While the A-LSTM network effectively learns temporal correlations, the estimation process can still introduce some new spatial errors or deviations from the actual posture. To counteract this, a least-squares distance constraint is adopted to refine the results estimated by the A-LSTM network. The least-squares fitting considers two frames, a reference frame and the estimated frame (frame where missing markers are estimated). In the reference frame, $P = \{P_i\}$ is a set of non-missing markers connected to one missing marker. i is the index of one marker. In the estimated frame, the points corresponding to the same markers are denoted $Q = \{Q_i\}$. Those two sets can be related by introducing a rotation matrix R , a translation vector T and a noise vector N_i , through the equation:

$$P_i = RQ_i + T + N_i$$

Thus, the least-square fitting finds the optimal R_s and T_s such that:

$$[R_s, T_s] = \arg \min_{R, T} \sum_{i=1}^n \|P_i - (RQ_i + T)\|^2$$

followed by:

$$M' = R_s M + T_s$$

with M the positions of the missing markers obtained through A-LSTM and M' the final coordinates after refinement by LS.

The results showed that A-LSTM+LS consistently achieved smaller RMSE than A-LSTM alone, further validating the utility of this spatial constraint. A-LSTM+LS method has proved to be robust to both the length of missing gaps and the number of missing markers.

2.2. GCN-based approaches

GCN are a particular type of Graph Neural Networks (GNN). As their name suggests, they take graph-structured data as input. Those networks are particularly relevant in this thesis because a skeleton is basically a graph. In our case, the nodes are the mouse’s body parts, their attributes are their 3D positions over time and the edges are the segments linking the body parts. GCN have become very popular since their first implementation and are widely adopted in skeleton-based HAR [36]. Therefore, we will now explore some GCN methods that seem suited to solve our problem.

2.2.1. Overview of ST-GCN Architecture

The original GCN

The original GCN computation by Kipf et al. [37] is:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right)$$

In this equation, $H^{(l)} \in \mathbb{R}^{N \times D}$ is the matrix of activations in the l^{th} layer with $H^{(0)} = X$ the input features. Besides, $\tilde{A} = A + I_N$ is the adjacency matrix of the undirected input graph with added self-connections (identity matrix), so that $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ is the degree matrix with added self-connections. It results that $\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$ is the normalized version of $\tilde{A}$. $W^{(l)}$ is the weight matrix for layer l and $\sigma(\cdot)$ is an activation function. This equation can be derived from spectral graph convolution with the help of some mathematical approximations. Kipf et al. successfully applied their network to semi-supervised learning tasks but since then the GCN has been applied to a wide variety of tasks.

ST-GCN

Yan et al. are the first ones to apply graph-based neural networks to model dynamic graphs over large-scale datasets. They tackle the task of human action recognition by designing the Spatial-Temporal Graph Convolutional Network (ST-GCN) [38]. ST-GCN is skeleton-based and addresses significant limitations of previous methods by automatically learning spatial and temporal patterns from data. This approach offers greater expressive power and stronger generalization capability compared to previous conventional techniques such as CNN and RNN, including LSTM, that rely on hand-crafted skeleton features, traversal rules, or simply ignores the skeletal structure.

The first stage of the ST-GCN pipeline is the graph construction. It takes a skeleton sequence, typically 2D or 3D joint coordinates over time, and transforms them into an undirected spatial-temporal graph $G = (V, E)$. Nodes (V) represent individual body joints at specific time frames, with input features consisting of joint coordinates and estimation confidence. In addition to the usual intra-body (spatial) edges, the spatial-temporal graph present connections of the same joint across consecutive frames, explicitly representing its trajectory over time. This construction allows for naturally defined connections without manual part assignment, making the architecture adaptable to datasets with varying numbers of joints or connectivity (e.g., 18 joints from OpenPose for Kinetics, 25 joints from Microsoft Kinect for NTU-RGB+D).

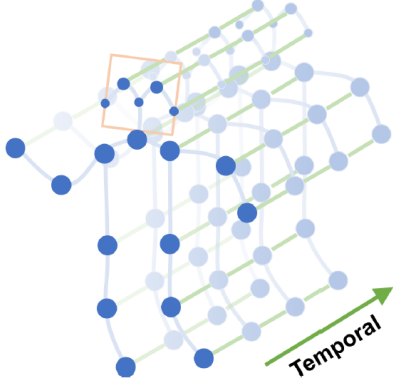


Figure 2.6.: Spatio-temporal graph [38]

The spatial graph convolution is reintroduced in a slightly different manner than the original GCN. This concept generalizes the 2D image convolution operation to graphs. For a given time t and given joint i , considering a node v_{ti} and its neighborhood $B(v_{ti})$ (including itself), a mapping function $l_{ti} : B(v_{ti}) \rightarrow \{0, \dots, K-1\}$ can be defined with K a number of fixed subsets. The values of K will be discussed below. Thus, the graph convolution can be written:

$$f_{out}(v_{ti}) = \sum_{v_{tj} \in B(v_{ti})} \frac{1}{Z_{ti}(v_{tj})} f_{in}(v_{tj}) \cdot w(l_{ti}(v_{tj}))$$

where f_{in} and f_{out} are the input and output feature map, w is the weight function (produces a different weight vector for every value of l_{ti}) and $Z_{ti}(v_{tj})$ is a normalizing term.

Furthermore, the design of the label map l_{ti} is crucial. ST-GCN explores several strategies for the label map. The first and simplest one is called uni-labeling, meaning all nodes in a neighborhood (including the root node) share the same label ($K = 1$). This is the classical aggregation technique used by Kipf et al. and that is efficient for node classification but suboptimal for human action recognition. The distance partitioning strategy is a bit more complex but still straightforward. Neighbors are divided into two subsets based on their distance to the root node: 0 for root and 1 for immediate neighbors ($K = 2$). This gives better representation of the skeleton graph. Finally, the spatial configuration partitioning is the most advanced strategy. It divides neighbors into three subsets ($K = 3$): the root node itself, “centripetal” nodes (closer to the gravity center of the skeleton than the root), and “centrifugal” nodes (further from the gravity center). This is inspired by concentric and eccentric motion patterns and this strategy achieves the best empirical performance. These three strategies are visible in Figure 2.7.

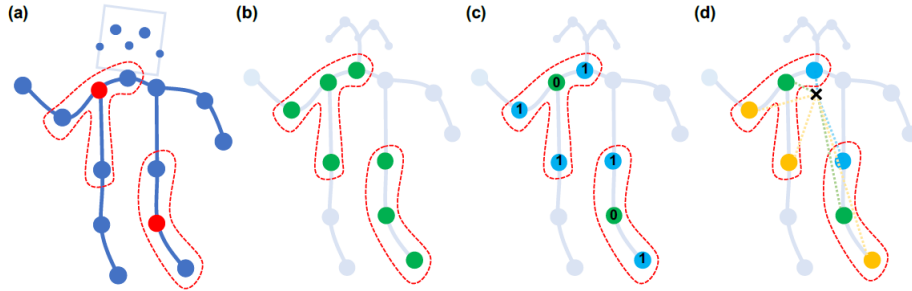


Figure 2.7.: Partitioning strategies [38]

There is also the possibility to incorporate a learnable mask (M) at each spatio-temporal graph convolution layer. This mask scales the contribution of a node’s feature to its neighbors based on a learned importance weight for each spatial graph edge. This further enhances recognition performance.

At last, the graph convolution of ST-GCN implementation for one layer can be described by the formula:

$$\mathbf{f}_{\text{out}} = \sum_{k=0}^{K-1} D_j^{-\frac{1}{2}} (A_k \odot M) D_k^{-\frac{1}{2}} \mathbf{f}_{\text{in}} W_k$$

where $D_k^{ii} = \sum_j A_k^{ij} + \alpha$ and α is set to 0.001 to avoid empty rows. In the $K = 0$ case, the formula becomes exactly the same as the Kipf et al. one (with $\tilde{A} \odot M$). The temporal convolution is a standard 2D convolution.

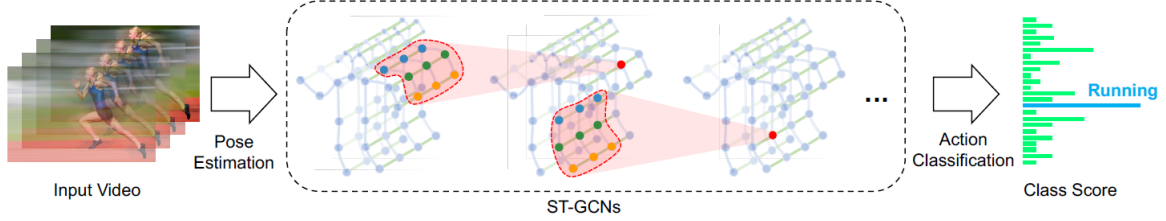


Figure 2.8.: ST-GCN framework [38]

ST-GCN comprises many layers (typically 9) of spatio-temporal graph convolution operators and incorporates residual connections, dropout for regularization, and temporal pooling layers. It terminates with a global pooling layer and a softmax classifier for action classification.

It demonstrates superior performance over then mainstream and state-of-the-art methods for action classification. It outperforms ST-LSTM+LS [9] on Kinetics and NTU-RGB+D 60. This demonstrates its robustness to illumination, scene variations, and different data acquisition methods.

2.2.2. Representative Methods

GCN Pose Refinement

Lino et al. employ ST-GCN for skeletal data reconstruction in their recent work 3D Human Pose Estimation (HPE) with Occlusions: Introducing BlendMimic3D Dataset and GCN Refinement [6] 3D HPE aims to give 3D positions of person from images and videos. This field is critical in computer vision because of its wide range of applications including HAR. Despite 3D HPE popularity, datasets including many occlusion scenarios are not common. That's why they propose a synthetic dataset for 3D HPE under occlusion that they call BlendMimic3D Dataset. In addition, they provide a GCN pose refinement network that enhances the joint positions predicted by 3D HPE networks like VideoPose3D [39] and PoseFormerV2 [40]. We will concentrate on this network.

The GCN-pose refinement module is designed as a plug-and-play solution that can be adapted to various 3D HPE frameworks without requiring them to be retrained. It starts with the same spatio-temporal graph construction as ST-GCN but with a novel partition strategy. Inspired by Cai et al.[41], joints have an additional connection to their symmetric joint (e.g, right elbow and left elbow) . The connected nodes are divided in 6 classes, that means one class per node in the neighborhood. It is easily interpretable in Figure 2.9.

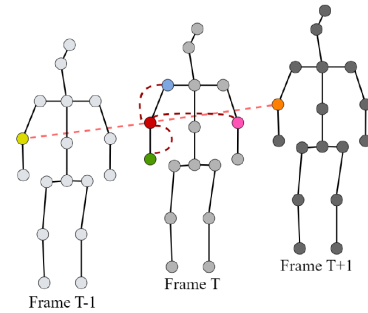


Figure 2.9.: Partition strategy [6]

2. Literature review

The network architecture comprises several spatio-temporal graph convolutional layers depicted in Figure 2.10. A non-local block that is not explicated is supposed to capture long-range dependencies along with body parts relationships.

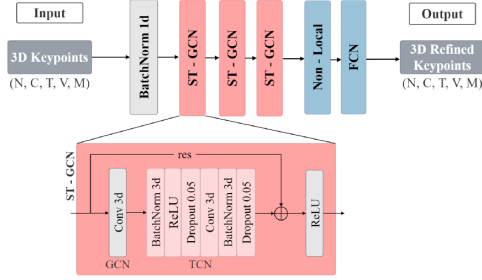


Figure 2.10.: Network architecture [6]

the symmetry loss that is the bone length difference between two symmetric bones (e.g left arm and right arm). They evaluate the network performance with the Mean Per-Joint Positional Error (MPJPE).

The integration of the GCN refinement block leads to significant improvements as shown in Figure 2.11. Indeed, applying the GCN module to the BlendMimic3D dataset yields an MPJPE approximately 30% lower. Additionally, it does not lead to a noticeable decrease in accuracy on the non-occluded Human3.6M dataset. This means that the network can be plugged to a 3D HPE algorithm safely.

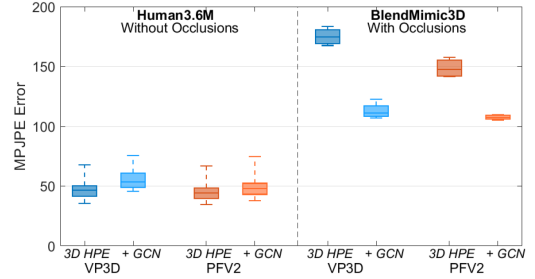


Figure 2.11.: Quantitative results [6]

Qualitative results further demonstrate the good performance of the GCN pose refinement network on occluded scenarios. In conclusion, this method reinforces the idea that GCN could be suited to addressing our problem.

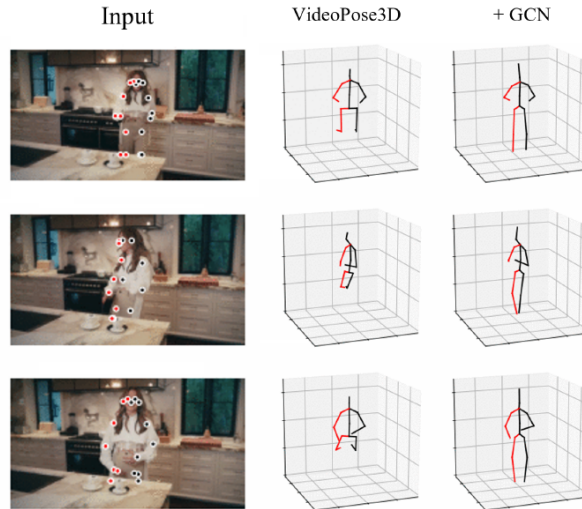


Figure 2.12.: Qualitative results [6]

2s-AGCN

We will now present a method that does not attend to occlusion or any kind of noise explicitly, yet it introduces innovative mechanisms and could be employed for reconstruction after some adjustments (modify final layers, adapt depth, etc...). This method is the Two-Stream Adaptive Graph Convolutional Network (2s-AGCN) [42] and was proposed by Shi et al. as a direct upgrade of the original ST-GCN for skeleton-based action recognition. According to occlusion experiments results presented in Table 2.1, 2s-AGCN is one of the most robust networks to occlusion among the GCN methods for skeleton-based HAR.

	Occluded part					
	None	Left arm	Right arm	Trunk	Two hands	Two legs
ST-GCN	71.53	57.64	55.56	66.67	59.03	54.86
2s-AGCN	90.28	77.08	61.80	81.25	75.69	81.25
MS-G3D [43]	93.75	80.56	60.58	82.72	79.17	75.42
ResGCN [44]	87.50	76.39	61.81	85.42	84.03	77.78

Table 2.1.: Recognition accuracy (%) under different occlusion scenarios [36]

Therefore, we will explore 2s-AGCN architecture to get a sense of how it can achieve these results. The first motivation to implement 2s-AGCN was that ST-GCN manually defines a graph structure only based on the physical skeleton and that is fixed in time. That means features of right and left hands are almost uncorrelated (because of their distance in the graph) even for some actions like “clapping” or even “walking”, which does not make a lot of sense and thereby seems non-optimal. Besides, the fixity of the spatial-temporal graph is also a problem not only going through the network layers but also going across the dataset samples (e.g., “wiping face” versus “jumping up” require different joint connection strengths).

As a ST-GCN successor, 2s-AGCN also uses its spatio-temporal graph construction with the spatial partitioning strategy. However, 2s-AGCN revises the graph convolution to make the graph into a learnable and flexible structure, hence the “adaptive” in its name.

To this end, the adjacency matrix is divided into three parts:

- A_k , that is the original normalized $N \times N$ adjacency matrix from ST-GCN, representing the physical connections of the human body.
- B_k , that is an $N \times N$ adjacency matrix whose elements are parameterized and optimized during training. The particularity of this matrix is that it has no constraints on its values. They are initialized to 0 and then, the model is able to learn entirely data-driven graph structures for the specific task for every layer of graph convolution. Besides, unlike the attention matrix M_k in ST-GCN, B_k can create new connections that do not exist in the original physical graph.
- C_k is a data-dependent graph. It learns a unique graph for each sample by calculating the similarity between the vertices in a learned embedding space. According to Xin et al. in their review about Transformers for skeleton-based HAR [45], this matrix is analogous to the transformer self-attention mask. C_k parameters are also initialized to 0.

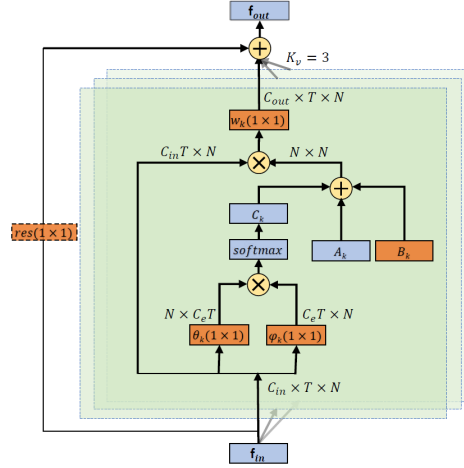


Figure 2.13.: Adaptive Graph Convolutional Layer (AGCN) [42]

The second innovation of 2s-AGCN is its two-stream framework. The goal of the second stream is to investigate the second-order information of skeletal data, such as the lengths and directions of bones. This information is naturally more informative and discriminative for action recognition. Simply, each bone is represented as a vector pointing from its source joint (closer to the center of gravity) to its target joint (farther away). To keep the same number of vertices as in the joint graphs, an empty bone directed from 0 the central joint is added. Each stream consists in a succession of the same number of adaptive graph convolutional blocks.

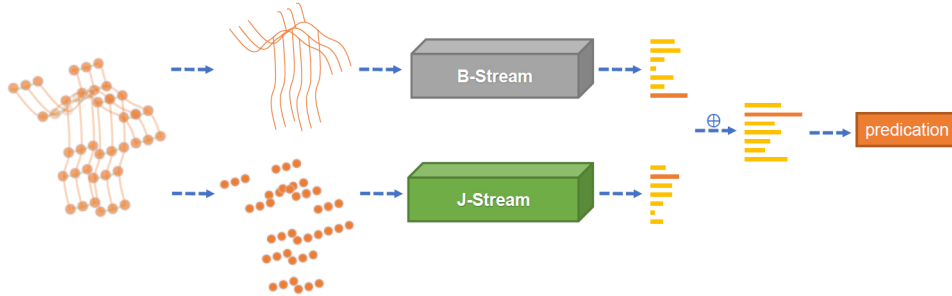


Figure 2.14.: 2s-AGCN framework [42]

2s-AGCN achieves superior performance compared to ST-GCN on NTU-RGB+D and Kinetics. Ablation studies show that every element introduced by 2s-AGCN results in an improvement of action recognition accuracy. After reviewing this method, it seems that the adaptive graph convolutional block is the key to its robustness to noise and occlusion. The adaptative matrices play an essential role by capturing new joint connections. Information can flow from and to every part of the skeleton as the situation demands. Therefore, missing joints have way less impact because reversed joints can still use one another to compute representative features during training. In the classical ST-GCN approach, missing joints can “cut” the information flow. In other words, they can greatly undermine the graph convolution. The very good results under occlusion of MS-G3D [43] and Res-GCN [44] seems to confirm this thought. Similar to 2s-AGCN, both are multi-stream methods (two streams for MS-G3D and three streams for Res-GCN) but the advanced strategies to connect their graphs probably contribute more to their ability to handle occlusions. MS-3GD proposes an actual spatio-temporal connectivity where one joint at a specific time can be connected to different joints at other times and Res-GCN integrates part-wise attention mechanism, meaning attention between joints regrouped in body parts. Now, suppose that we train ST-GCN and 2s-AGCN to recover one synthetically occluded joint

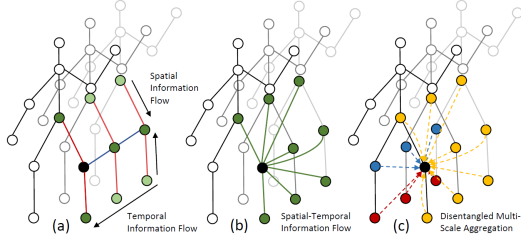


Figure 2.15.: MS-G3D partition strategy [43]

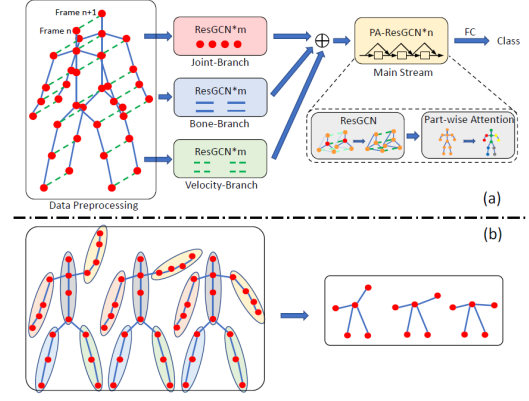


Figure 2.16.: Res-GCN framework [44]

coordinates. With ST-GCN, the occluded joint coordinates would be mostly recovered from its immediate neighborhood while with 2s-AGCN, the coordinates could be inferred from any joint of the same frame. For instance, imagine the right-hand joint is occluded for a “walking” action. 2s-AGCN would find connections between the right and left hands whereas ST-GCN would mostly rely on right arm joints (elbow, then shoulder ... in decreasing order of importance). Every human being can agree that the position of the left hand, along with the legs joints positions, also give substantial information about the position of the right hand in the “walking” action. Overall, 2s-AGCN should give better results than ST-GCN in the task of skeleton reconstruction.

2.3. Transformer-based methods

Originally implemented for NLP, the groundbreaking performance of Transformers lead researchers to adopt it across various other domains and particularly in computer vision. HAR did not escape the wave of Transformers. According to Xin et al. [45], they should become the mainstream methods in this field in the near future, definitively replacing the GCN. Transformers have the advantage to purely rely on the self-attention mechanism, making them largely data-driven. This implies that Transformers acquire representations and features directly from data, while depending only minimally on architectural inductive biases. Thus, compared to LSTM, Transformers better capture global (long-term) temporal and obviously spatial dependencies. Compared to GCN, they better compute non-physical joint correlations (in spatial and temporal dimensions). However, this comes to the price of understating local neighborhoods (because both global and local information are treated equally).

2.3.1. Overview of Transformer Architecture

The architecture was first proposed by Vaswani et al. in the paper *Attention is all you need* [46]. Figure 2.17 highlights the encoder-decoder structure of the Transformer. Both the encoder and decoder are composed of a stack of N identical layers and used similar blocks, notably the attention and positional encoding (along with the embedding) that will be detailed below. All sub-layers and embedding layers produce outputs of dimension d_{model} . The systematic residual connections followed by layer normalization are also visible on the figure.

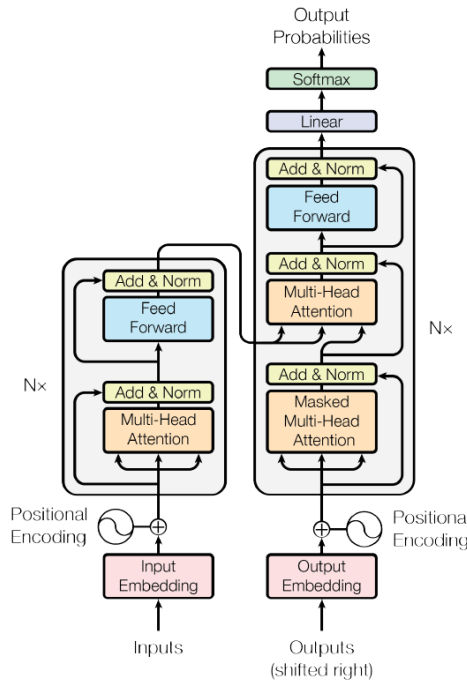


Figure 2.17.: Transformer architecture [46]

As mentioned in the introduction, in the core of Transformer mechanism lies the multi-head attention. The attention computation used in Transformer is called Scaled Dot-Product Attention with the equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where Q is the matrix of the queries, K is the matrix of the keys and V is the matrix of the values. d_k is the common dimension of queries and keys while the dimension of values is d_v . The dot product acts as a similarity metric and the $\frac{1}{\sqrt{d_k}}$ scaling factor enables an acceleration of the calculation. The softmax function gives weights for the values.

The Multi-Head Attention mechanism consists in computing several Scaled Dot-Product Attention but in parallel and without increasing d_{model} . To this end, queries, keys, and values are linearly projected h times (with different learned projections) to lower dimensions. d_k and d_v are equal to $\frac{d_{model}}{h}$ and attention is performed on the projected matrices. After concatenation, the outputs are projected one last time. Mathematically, we have:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{where } \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

where W_i^Q , W_i^K and W_i^V respectively are the projection matrices of the queries, keys and values for the head i . W^O is the output projection matrix.

Thanks to the Multi-Head Attention, the network can learn different representational subspaces in parallel in the same manner that parallel kernels capture distinct features in CNN. Now going back to the architecture, we notice three different kinds of multi-head attention:

- the encoder self-attention: keys, values, and queries come from the output of the previous encoder layer.
- the decoder self-attention, also called masked self-attention: keys, values, and queries come from the output of the previous decoder layer, but here, every position can only attend to previous positions. This is done by masking with -infinity all coefficients corresponding to prohibited connections in the scaled dot-product matrix.
- the encoder-decoder attention: queries come from the previous decoder layer, while keys and values come from the output of the encoder. Every position in the decoder can attend to every position in the input sequence.

Another crucial element of the Transformer is the embedding of the entries associated with a positional encoding. As a matter of fact, Transformers have no built-in sense of order as LSTM which process elements one-by-one in a sequence, thereby the need for a positional encoding to indicate to the network the positions within a sequence. A sequence is composed of tokens and their position (pos) in the sequence is encoded as such:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin\left(pos/10000^{2i/d_{model}}\right), \\ PE_{(pos, 2i+1)} &= \cos\left(pos/10000^{2i/d_{model}}\right) \end{aligned}$$

Here, i denotes the index of a pair of dimensions, meaning i maximum value is $\left\lfloor \frac{d_{model}}{2} \right\rfloor$.

2. Literature review

The unicity of the sinusoidal positional encoding is assured for input with sequence length inferior to $T = 2\pi * 10000$. Indeed, the last pair of dimension varies slowly and has a period T , meaning the encodings of these dimensions cannot be the same for two positions inferior to T , hence the global unicity of the sinusoidal positional encoding. Besides, it also shows that the first dimensions (with high frequency) capture the local variations and the last ones (with low frequency) capture the global order. One last benefit of using the sinusoidal positional encoding is the efficient computation of a relative position $pos + k$ (k is a fixed integer) by linear trigonometric formulas. It is possible to replace the positional encoding by learned positional embedding, yielding similar results in the experiments of this paper. This proves that Transformers can automatically capture temporal dependencies but the authors still chose to keep the positional encoding given its many advantages. The positional encodings are added to the entries (input and output during training) embeddings. Embeddings are computed through learnable simple linear layers and the input, output and final decoder layers (“un-embedding layer”) all share the same weights.

Finally, while establishing better or similar results than previous state-of-the-art on translation tasks, Transformers also require significantly less time to train. This is a consequence of self-attention connecting all positions with a constant number of sequential operations ($O(1)$), unlike recurrent layers ($O(n)$) or convolutional layers ($O(\log k(n))$ or $O(n/k)$). That is why Transformer can easily learn long-range dependencies.

2.3.2. Representative Methods

Skeleton MAE

When presenting LSTM-based approaches, we discussed DGAE. More recently, He et al. proposed the Masked Autoencoder (MAE) [47] that actually is a particular type of DGAE. They use ViT as the encoder and a simple Transformer with less parameters than the ViT as the decoder. MAE is characterized by its high proportion of masking on the images. Wu et al. built an MAE method for skeleton-based action recognition, namely SkeletonMAE [3]. By pre-training on unlabeled data to reconstruct masked skeleton sequences, the model learns comprehensive and generalizable spatial and temporal features. After that, a fully connected layer is added on top of the trained encoder and this network is fine-tuned for action recognition on all the training data. This is self-supervised learning.

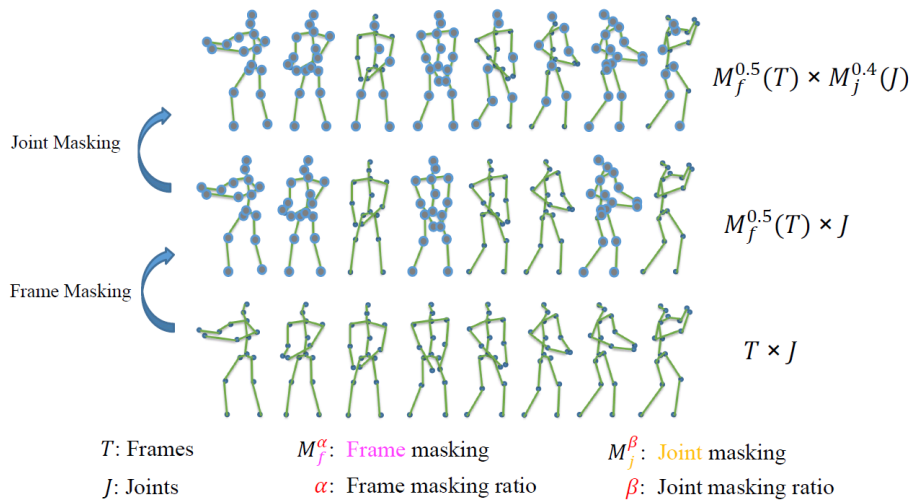


Figure 2.18.: SkeltonMAE masking strategy [3]

The principal contribution of SkeletonMAE is the development of a spatio-temporal masking strategy. Firstly, a random subset of entire frames is removed from the sequence based on a pre-set frame-masking ratio. This is the temporal masking. Then, within the remaining frames, a random subset of joints is masked based on a pre-set joint-masking ratio. Figure 2.18 depicts this spatio-temporal masking. Subsequently, the masked skeleton sequence is fed to the encoder that outputs its representation, followed by the decoder which interprets this representation to output the recovered skeleton sequence. Both the encoder and the decoder are based on the Spatio-Temporal Tuples Transformer (STTFormer) [48] that was already developed for skeleton-based action recognition.

In contrast to GCN methods, STTFormer enables connection between different joints in adjacent frames as illustrated in Figure 2.19. Indeed, those connections can be important for the motion reconstruction as depicted by the “long jump” example, where arms position in a previous frame give crucial information about legs in the next frame.

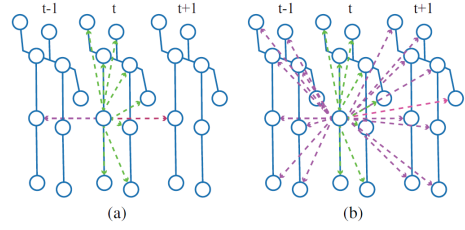


Figure 2.19.: Typical GCN vs STTFormer self-attention schemes [48]

To be able to build such connections, STTFormer introduces an innovative method with tuples of skeleton sequences. The skeleton sequences are processed as shown in Figure 2.20. C_0 , T_0 and V_0 respectively denote the original number of input channels, the total length of the sequence (in number of frames) and the number of joints in the skeleton. C_0 is augmented to C_1 thanks to a feature mapping layer and the sequence is divided in $T = \frac{T_0}{n}$ non-overlapping tuples, meaning each tuple contains n consecutive frames. Then, those tuples are flattened with $V = n \times V_0$ and a convolutional layer embeds the tuples to C channels. The positions of the joints in this flattened embedded sequence is encoded with a sinusoidal positional encoding (here $d_{model} = C$).

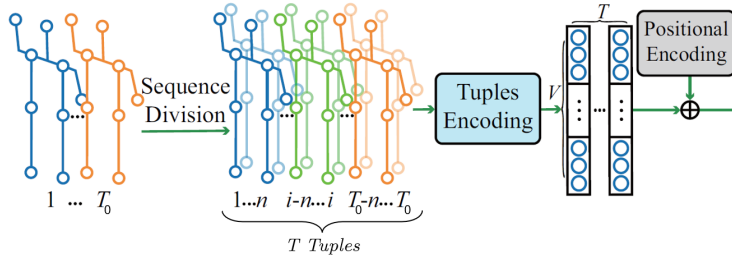


Figure 2.20.: STTFormer data preprocessing [48]

Now, Spatio-Temporal Tuples Attention (STTA), that is mutli-head self-attention with minor adjustments, can be applied. Concretely, with X_{in} denoting the embedded sequence augmented with positional encoding, the self-attention is computed in this way:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \text{Conv2D}^{(1 \times 1)}(\mathbf{X}_{in})$$

and,

$$\text{Attention}(Q, K, V) = \tanh\left(\frac{QK^\top}{\sqrt{C}}\right) V$$

2. Literature review

Then, with the same definition of projection matrices as the original Transformer, we have:

$$X_{attn} = \text{Concat}(\text{head}_1, \dots, \text{head}_h),$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

and finally, the output projection is carried out by a 2D convolution of kernel size $(1 \times k_1)$:

$$X_{STTA} = \text{Conv}_{2D(1 \times k_1)}(X_{Attn})$$

with k_1 an hyperparameter of the network. It controls the number of neighboring joints to agglomerate for the projection.

Moreover, the initial sequence was divided in shorter sequences but correlation between those short sequences can also be essential to recover the action represented in the input. For instance, a “long jump” can be divided in sub-actions such as “run-up”, “take-off” and “landing”. If one sub-action is represented by one tuple, the model can retrieve the “long jump” action by associating the tuples, hence the Inter-Frame Feature Aggregation (IFFA). It is computed as follows:

$$X_{IFFA} = \text{Conv}_{2D(k_2 \times 1)}(X_{STTA})$$

with k_2 an hyperparameter of the network controlling the number of tuples to aggregate.

Now going back to SkeletonMAE, we can observe the overall architecture in Figure 2.21. Note that the encoder and the decoder do not have to be symmetrical. In MAE, the decoder is often designed to be “smaller” than the encoder. The autoencoder is trained to minimize MSE loss between the reconstructed skeleton sequence and the original input sequence.

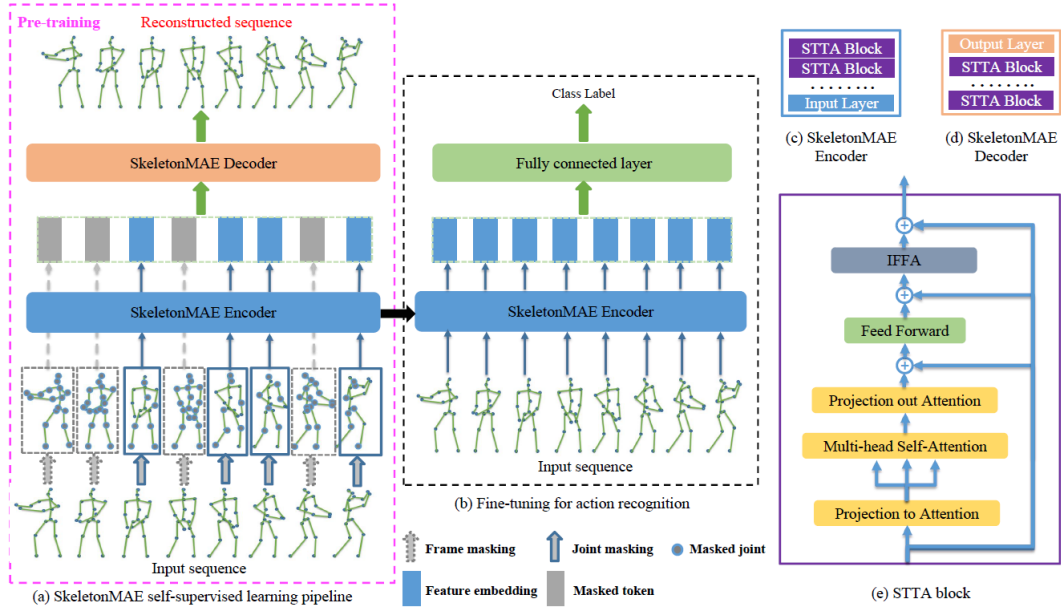


Figure 2.21.: SkeletonMAE framework [3]

Finally, configuration experiments are conducted to find the optimal masking ratios for action classification accuracy. It results that $\alpha = 0.4, 0.5$ and $\beta = 0.5$ yield the best results. With those ratios, SkeletonMAE consistently outperforms previous self-supervised methods on the NTU RGB+D 60 dataset, especially when fine-tuning with only 5 or 10 % of the available data. However, it still performs worse than 2s-AGCN. Besides, no quantitative result is given for the reconstruction but visualization show that the skeletons are effectively retrieved.

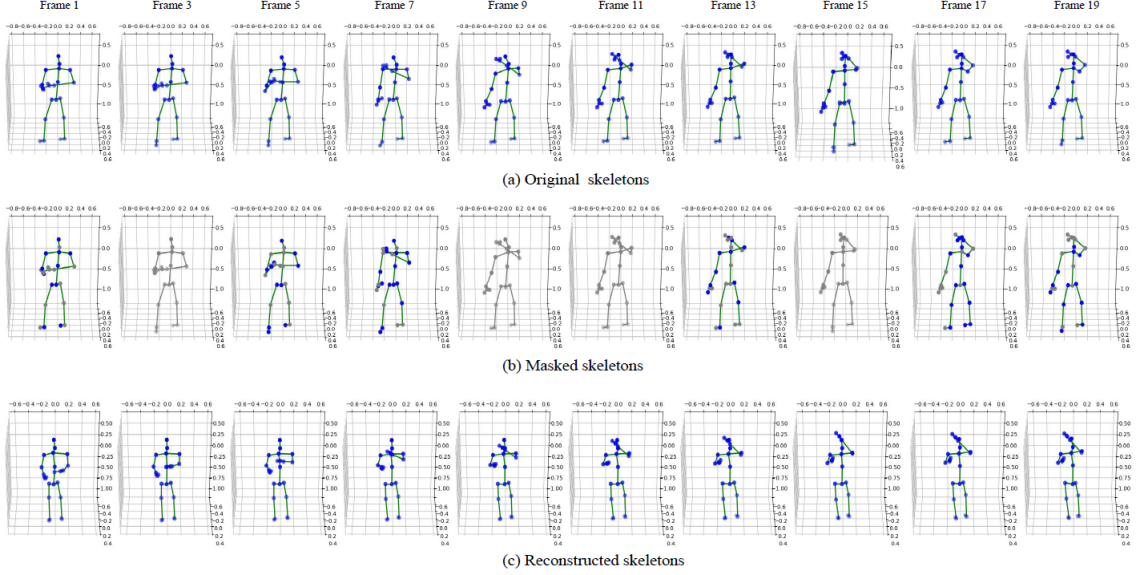


Figure 2.22.: SkeletonMAE skeleton reconstruction [3]

Thus, SkeletonMAE proves that skeletal data reconstruction can be performed with Transformers, and more particularly with Spatio-Temporal Transformers. Besides, this reconstruction can be achieved even in very occluded situations thanks to the ability of STTFormer to connect data in a large spatio-temporal space. As for 2s-AGCN, the flexibility of the structure enables a robustness to noise.

3. Methodology

This section will introduce the tools used during the thesis to implement a DL solution to the occlusion problem of body parts of a mouse in a lockbox experiment. As mentioned in the introduction, we work with a simulated data set for simplicity and in-depth analysis. Furthermore, only the trajectory of a single mouse is generated, so that we can purely focus on the occlusions. Those occlusions are also artificially generated and will be described below. It will be followed by a section reporting on the selection process of the DL networks. We evaluate them thoroughly in the next chapter.

3.1. Dataset

3.1.1. Data Generation [1]

The dataset consists of the rigid and deformable 3D coordinates of the body parts. We let the rigid mouse model move along a 2D random curve (random walk/Brownian motion ...) on a quadratic plane filling the field of view of cameras. The mouse consists of ten body parts moving jointly. Those body parts are listed in Table 3.1 with an integer ID between 0 and 9. The two last body parts (head center and tail base), that are actually joints strictly speaking, are inferred from other body parts. The head center is the midpoint between the two ears, while the tail base is the midpoint of the hind paws but at head center's height.

The mouse travels in the (X, Y) plan. During the model movement along the curve, the body parts deform plausibly. They move according to pace: while right front and left hind paw are fixed to the ground, left front paw and right hind paw move forwards twice as fast as the overall mouse body. Meanwhile the rigid head triangle consisting of nose tip and ears rotates linearly back and forth in three limited angle intervals around the mid point of the line connecting the two ears. Figure 3.1 depicts the deformable movements of the mouse's paws.

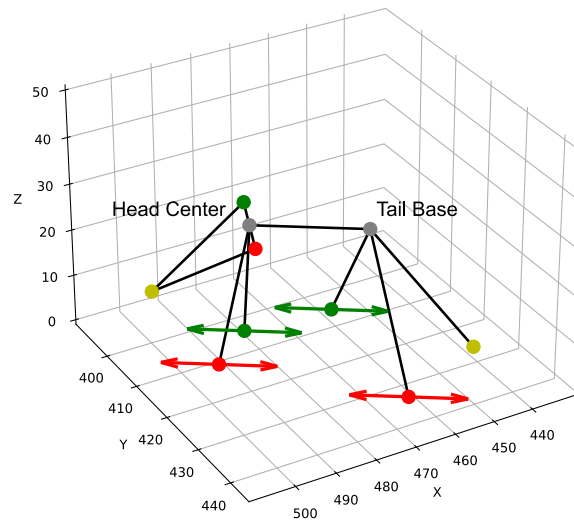


Figure 3.1.: Mouse model

3.1.2. Statistics

body part	X [mm]	Y [mm]	Z [mm]
0: nose tip	0	36	2.5
1: left ear	7.75	16	19
2: right ear	-7.75	16	19
3: left front paw	5.5	20	-8
4: right front paw	-5.5	20	-8
5: left hind paw	13.5	-8.5	-8
6: right hind paw	-13.5	-8.5	-8
7: tail tip	0	-30	-6
8: head center	0	16	19
9: tail base	0	-8.5	19

Table 3.1.: Biologically realistic rigid mouse model in mouse’s coordinate system

The dataset has a rate of 30 frames per second (fps). For every body part, there is $T_0 = 4998$ time steps, which means that the mouse walks for $(4998 - 1)/30 \approx 166.57s$. The height (27 mm) and length (66 mm) of the mouse can be easily inferred from Table 3.1. The mouse freely moves in a space with the following dimensions $\Delta X \approx 600$ mm, $\Delta Y \approx 600$ mm and $\Delta Z = 27$ mm. The height of the mouse is equal to ΔZ because the mouse does not move at all along the Z axis, whether in the rigid or the deformable model.

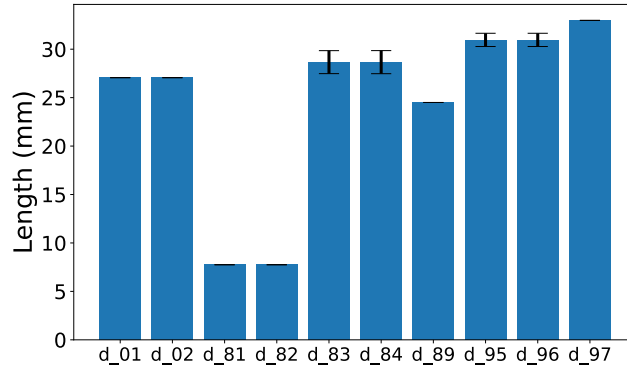


Figure 3.2.: Mean bone lengths with standard deviation for one sequence of 100 time steps

Additionally, we calculate the lengths of the mouse’s bones and display them in Figure 3.2, along with their standard deviations in a single sequence of 100 time steps. d_{ij} denotes the distance from part i to part j . The standard deviations of the bone lengths are all really close to 0 except for d_{83} , d_{84} , d_{95} and d_{96} ; that are closer to 1 mm. This is not surprising as these lengths correspond to the front paws to head center and hind paws to tail base edges. Given that no part moved in the Z direction, the back and forth movement of one paw relative to its joint modifies the edge length between them.

3.1.3. Masking

In the rest of this thesis, we use N for the batch size, T is the temporal dimension, $V = 10$ is the number of body parts and $C = 3$ is the number of spatial dimensions. The dataset actually comes in the shape $(T_0 \times V, 2 \times C)$. Before feeding it to a network, we reshape it into $(T, V, 2 \times C)$ sequences where T is a fixed number of time steps. While experimenting with rigid and deformable coordinates, the results obtained were difficult to interpret so we discarded the rigid coordinates. Then, the DL networks are trained with (N, T, C, V) tensors as inputs.

We introduce occlusions by masking coordinates in three different manners that we call Coordinates, Parts and Frames and that we explicit just after. All the maskings are done by substituting the original coordinates with -1 . Given that coordinates can have negative values, we offset them so that all values are positive prior to processing. Note that all random processes mentioned follow discrete uniform distributions.

Coordinates

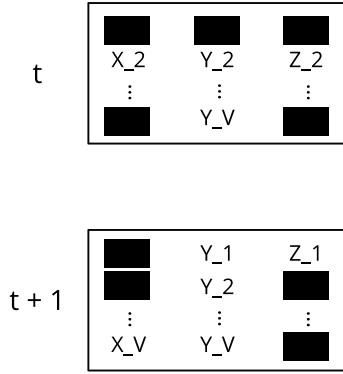


Figure 3.3.: Coordinates masking

Here we occlude random coordinates across the dataset with a missing rate (probability) p going from 0 to 0.8 (with a step of 0.2). The case $p = 0$ tests the ability of the network to learn the identity function. Incidentally, $p = 0$ is not specific to Coordinates, as it also represents the case where no parts and no frames are masked at all. For a single body part and for a given frame x_d and y_d can be occluded while z_d has its original value for instance. This masking does not really translate to a real situation but it gives us a general idea of the model performance. Indeed, we noticed that this masking is the most stable one, in the sense that training the same model many times with this masking will give very similar results.

Parts

For this masking we occlude coordinates for some body parts for a whole sequence, except for the five first and last time steps of the sequence. This ensures that a part cannot be totally occluded for longer than one sequence length. The number of masked body parts for a sequence varies between 1 and 4. Those body parts are selected randomly. It can depict a situation where some body parts are occluded for a long period of time whereas other body parts are perfectly visible. Thanks to this masking we evaluate the model's ability to infer deformable coordinates for a body part relying on the other body parts.

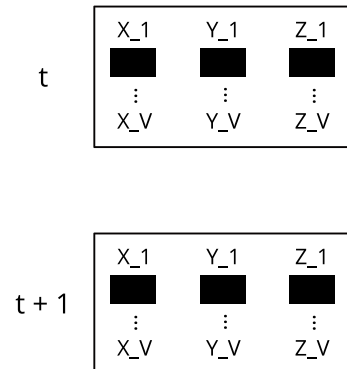


Figure 3.4.: Parts masking

Frames

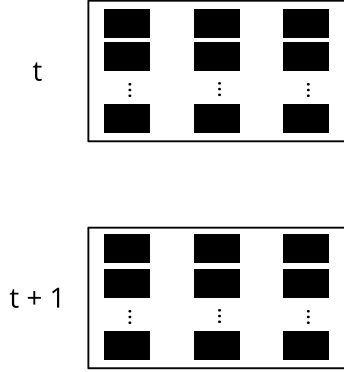


Figure 3.5.: Frames masking

We mask contiguous frames in every sequence. The number of masked frames for a sequence is either 10, 20 or 30 frames (respectively, 1/3, 2/3 and 1 s). The starting time step of this succession of masked frames is selected randomly but in a range that excludes the the end of the sequence. That is, with *length* the number of masked frames, the series of masked frames is bounded between *sequence_start* and *sequence_end* – *length*. This ensures that the number of contiguous masked frames is actually the one we defined. This masking corresponds to a scenario where the mouse would go hiding below an element of the environment for instance but only briefly.

The python script below displays the effective implementation of the maskings through the *masking* function. It takes the original dataset as input as well as the *mask_length*, the *mode* and the *drop_rate*. *mode* is an ID for the masking types (0 for Coordinates, 1 for Parts and 2 for Frames) and *drop_rate* range of values changes based on the mode chosen. *drop_rate* is the missing rate for Coordinates, the number of occluded body parts for Parts and the number of occluded frames for Frames. All the maskings, are based on a fixed *mask_length* number of time steps, meaning that the function consider (N, *mask_length*, V, C) arrays consecutively without any overlapping. We fix *mask_length* to 100. Thus, Parts mask body parts for 90 timesteps = 3s. The value of *mask_length* has no influence on the Coordinates masking but it has a real importance for the Frames masking. A value of *mask_length* many times superior to 30 (maximum number of consecutive frames masked) guarantees that every masked sequence is surrounded by an enough amount of non-masked frames. It enables the DL networks to have enough data to infer the missing frames. Besides, the number of masked frames can also be interpreted as the total percentage of masked frames. Lastly, to ensure all sequences have the same temporal length and are masked in the same way, we discard the last 98 time steps, meaning $T_0 = 4900$ time steps throughout all the experiments.

```

1  import torch
2  import pandas as pd
3  import numpy as np
4
5  V = 10 # number of body parts
6
7  def masking(data, mask_length, mode, drop_rate):
8      masked = np.empty((0, data.shape[1]))
9      data_copy = data.copy(deep=True)
10
11     for i in range(len(data)//(mask_length*V)):
12         rng = np.random.default_rng(seed=42+i)
13         seq = data_copy.iloc[i*mask_length*V:(i+1)*mask_length*V,:].values
14         seq = seq.reshape(mask_length,V,-1)
15
16         # Coordinates
17         if mode == 0:
18             mask = rng.random(seq.shape) < drop_rate # drop_rate in [0, 0.2, 0.4, 0.6, 0.8]
19             seq[mask] = -1
20
21         # Parts
22         if mode == 1:
23             mask_idx = rng.choice(np.arange(V),size=drop_rate,replace=False) # drop_rate in [1, 2, 3]
24             seq[5:-5, mask_idx, :] = -1
25
26         # Frames
27         if mode == 2:
28             length = drop_rate
29             values = np.arange(0, mask_length - 2*length + 1)
30             idx = rng.choice(values)
31             seq[idx:idx+length, :, :] = -1
32
33         seq = seq.reshape(-1, 3)
34         masked = np.concatenate([masked, seq])
35
36     return masked

```

Thereby, our experiments include a significant amount of randomness. In addition to the maskings' random nature, we have to cope with the randomness inherent and present at almost any step of a DL network training, like the weight initialization and the optimization path for instance. Obviously, this hinders the analysis and reliability of the results. To mitigate the effects of randomness, we fix the initial random seed (highlighted in yellow in the Python script) of the masking processes for the training of all our networks. We also apply supplementary measures to address randomness that will be mentioned in the following sections

3.2. Exploration

We train DL networks to reconstruct the artificially corrupted data, so that all the networks aim to minimize the MSE loss:

$$MSE = \frac{1}{M} \|Y - \hat{Y}\|_2^2$$

with Y the original sequence of body part positions and $\hat{Y}$ its reconstruction by the network from its masked version. M denotes the total number of coordinates in the sequence. The norm is applied to the last dimension (that is the spatial dimension here) so the MSE is per body part.

For a thorough analysis of the networks' behaviors, we train one network for every *drop_rate* of every masking type. Those networks are then evaluated on a dataset with the same masking settings. This results in graphs similar to Figure 3.6.

In addition to the offset to positive values, we also normalize all coordinates into the range $[0, 1]$. This stabilizes the networks' optimization and leads to better results. This normalization is carried out for each coordinate dimension separately, meaning that $x_{min} = y_{min} = z_{min} = 0$ and $x_{max} = y_{max} = z_{max} = 1$.

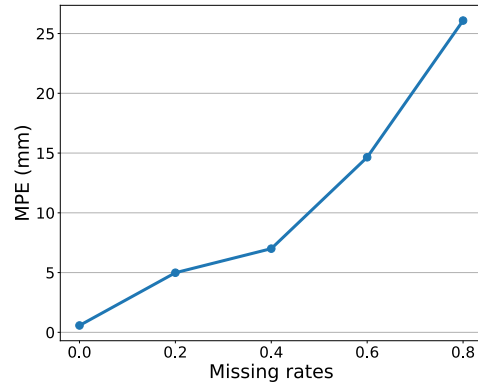


Figure 3.6.: Example of a result curve

The training and validation sets are fixed by a constant random seed in a 0.8-0.2 ratio. All networks are trained on single GPU from the High Performance Computing-Cluster (HPC-Cluster) of the TU Berlin. 4GB of RAM were requested every time. The networks are implemented with PyTorch[49] and the training are monitored with Lightning.

3.2.1. Metrics

We quantitatively evaluate our models with three different metrics, enabling us to characterize the efficiency of the models.

Mean Positional Error

The Mean Positional Error (MPE) is equivalent to the MPJPE from Lino et al. [6]. It is calculated in this way:

$$MPE = \frac{1}{M} \|Y - \hat{Y}\|_2$$

It is actually the MSE but without the square on the norm. This is our key metric as it is a direct measurement of the reconstruction error. It is implemented as follows:

```

1  import torch
2  import numpy as np
3
4  def paired_euclidean_distances(tensor1, tensor2):
5      return(torch.norm(tensor1 - tensor2, dim=-1))
6
7  mpe = 0
8
9  for x, y in val:
10     x, y = x.to(device), y.to(device)
11     pred = lit_model(x) # inference with the trained model; pred, x and y are normalized
12
13
14     y_nnorm, pred_nnorm = y * maxima, pred * maxima # maxima = [x_max, y_max, z_max]
15     # before normalization
16     distances = paired_euclidean_distances(y_nnorm, pred_nnorm)
17
18     # mpe
19     distances_mean = distances.mean(dim=(0, 1)) # Mean over batches and sequences
20     mpe += np.array(distances_mean) / len(val)

```

Therefore, one MPE per body part is computed for every *drop_rate*. We can present the results as in Table 3.2 (exemple for Coordinates) but we prefer displaying the mean MPE in a result curve as in Figure 3.6 for concision.

Missing rates	nose	left ear	right ear	left front paw	right front paw
0.0	0.77	0.76	0.45	0.65	0.53
0.2	4.92	3.54	3.22	6.41	5.83
0.4	7.06	5.19	5.32	7.69	8.05
0.6	17.26	10.15	10.89	12.27	12.85
0.8	33.12	23.50	24.72	25.57	26.66

Missing rates	left hind paw	right hind paw	tail tip	head center	tail base
0.0	0.54	0.54	0.63	0.47	0.40
0.2	7.04	6.44	5.79	3.18	3.50
0.4	9.11	8.78	8.31	4.97	5.54
0.6	16.75	17.79	24.92	9.39	14.21
0.8	24.33	26.24	31.12	23.35	22.24

Table 3.2.: Example of MPE values (mm)

Percentage of Correct Keypoints

The percentage of Correct Keypoints (PCK) relies on the MPE. For a given radius and a given body part, it classifies the reconstructed part as correctly retrieved if it is within the radius and as an error of prediction if it is not. We use a very restricted radius of 5 mm because it is roughly 10% of the dimensions of the mouse and strictly inferior to the minimum bone length (around 8 mm visible in Figure 3.2). It allows us to decide between networks when the MPE are similar. Below is its implementation and an example table.

```

1  ... # same implementation as mpe
2  counts = 0
3  counts_total = 0
4
5  for x, y in val:
6      ... # same implementation as mpe
7
8      # pck
9      correct = distances < 5
10     counts += np.count_nonzero(correct, axis=(0, 1))
11     counts_total += distances.shape[0] * distances.shape[1]
12
13 percentages = (counts / counts_total) * 100

```

Missing rates	nose	left ear	right ear	left front paw	right front paw
0.0	99.78	99.84	99.82	99.83	99.66
0.2	67.89	82.81	85.87	43.46	49.67
0.4	46.68	63.50	61.55	35.18	32.35
0.6	7.89	25.47	20.54	16.79	15.14
0.8	2.35	5.79	5.19	5.04	3.80

Missing rates	left hind paw	right hind paw	tail tip	head center	tail base
0.0	99.78	99.82	99.76	99.85	99.84
0.2	38.78	45.95	57.44	87.04	82.07
0.4	26.03	28.65	42.61	65.85	61.69
0.6	8.45	7.44	3.80	29.56	11.42
0.8	4.75	3.74	2.83	6.39	5.61

Table 3.3.: Example of PCK (%)

Bone-Length Error

Our last metric is the Bone-Length Error (BLE) coming from the BRA [7]. It quantifies how much the lengths of the prediction's bones differ from the ground-truth length. More generally, it expresses the ability of the model to keep the structure of the mouse's skeleton. In one batch, all the predicted bone lengths are compared to the very first corresponding ground-truth bone length (because this length is uniform or almost uniform) . Again, we display its implementation and an example table of values.

```

1  ... # same implementation as mpe
2
3  ble_01 = 0
4  ... # initialization of other ble
5
6  for x, y in val:
7      ... # same implementation as mpe
8
9      # Parts ground truth
10     nose_y = y_nnorm[0, 0, 0, :]
11     l_ear_y = y_nnorm[0, 0, 1, :]
12     ... # definition of other parts
13
14     d_01 = paired_euclidean_distances(nose_y, l_ear_y)
15     ... # definition of other distances
16
17     # Parts prediction
18     nose_p = pred_nnorm[..., 0, :]
19     l_ear_p = pred_nnorm[..., 1, :]
20     ... # definition of other parts
21
22     ble_01 += float(abs(paired_euclidean_distances(nose_p, l_ear_p)
23                  - d_01).mean()) / len(val)
24     ... # calculus of other ble

```

Missing rates	d_01	d_02	d_81	d_82	d_83	d_84	d_89	d_95	d_96	d_97
0.0	0.27	0.35	0.41	0.27	1.40	1.26	0.33	0.79	0.78	0.19
0.2	1.62	1.63	0.79	0.82	1.35	1.15	2.21	1.33	1.33	1.22
0.4	1.96	1.99	1.00	1.04	1.37	1.11	2.91	1.44	1.44	1.56
0.6	5.35	5.35	2.81	2.78	1.64	1.28	8.85	2.51	2.49	4.21
0.8	5.79	5.83	3.06	3.09	1.63	1.32	9.69	2.63	2.64	4.62

Table 3.4.: Example of BLE values (mm)

3.2.2. Networks

LSTM

The LSTM is our doorway to the comprehensive analysis of the dataset. As mentioned in the literature review, LSTM are a reference when working with temporal data, so we use this network to gain a better understanding of the problem. In practice, we systematically make hyperparameters vary and examine how it reflects on the LSTM performance. Ultimately, we get to a combination of hyperparameters that optimizes the LSTM results on our reconstruction task. Most of those hyperparameters are tested in a successive and not in a crossed manner as this would demand excessive computation time due to the large number of hyperparameters and their large range of values. We select the values of hyperparameters according to the performance on Coordinates. Owing to its stability in training, it enables us to have a higher degree of confidence in the hyperparameter selection compared to the other maskings without training the same networks many times. We use the MPE to discriminate the models.

All the LSTM are trained for a maximum number of 1000 epochs regulated by an Early Stopping callback with a patience of 100 epochs. This means that the training proceeds until the validation loss reaches a plateau for at least 100 epochs straight. 1000 can seem a bit extreme as a number of maximum epochs but the limited size of the dataset encourages us. It ensures that the LSTM has “time” to learn most of the temporal dependencies of the dataset. A patience of 100 for the Early Stopping can also be considered very high but it is still in the typical range 5-10% of the maximum number of epochs. We use the Adam optimizer [50] with an initial learning rate of 0.001 and we use ReduceLROnPlateau [51] as the scheduler.

Now, we can present some of the results obtained on the validation set while searching for the optimal hyperparameters. First of all, the networks are fed tensors of shape (N, T, V, C) but LSTM only considers the temporal dimension of the sequences, so we reshape the tensors to $(N, T, V \times C)$. After the LSTM cell, the built-in PyTorch LSTM linear projection head outputs $(N, T, V \times C)$ tensors that are reshaped back to (N, T, V, C) to calculate the training loss. We define $N = 128$ throughout all the experimentation. For the first experience, $T = 100$ time steps and the *stride*, that is the starting time step difference between two successive sequences, is equal to 1. This means that two consecutive sequences in the batch share 99 common time steps.

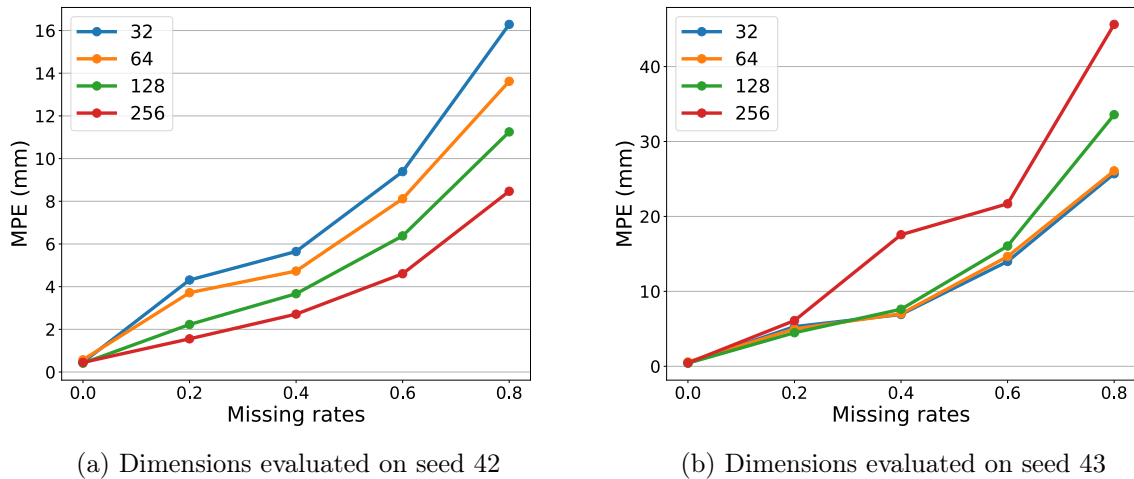


Figure 3.7.: Embedding Dimensions

We start with a simple LSTM network and we make the embedding dimension vary. Figure 3.7 showcases the evaluation of the LSTM on the validation set with the same random seed used during training (3.7a) and the evaluation on the same validation set but with another random seed (3.7b). Standard ML/DL procedures recommend to choose hyperparameters based on the results observed on the validation set used during the training phase, however it makes more sense to choose according to the other validation set. On the one hand, Figure 3.7a just proves that increasing the dimension of the LSTM enables the better recovery of the same missing coordinates, the MPE decreasing almost linearly with the embedding dimension. On the other hand, Figure 3.7b is way more interesting because it demonstrates that at a very high missing rate, a LSTM with low embedding dimension (32, 64) performs significantly better than one with a higher embedding dimension (128, 256). We decide to keep $embed_dim = 64$ for the remainder of the LSTM experiments, even though $embed_dim = 32$ exhibits similar results (also for PCK). The reason is that a higher embedding dimension enables a better features representation. These figures also reveal an overfitting of the LSTM to the original masking of the validation set. This is clearly illustrated by the lower range of MPE values on the original validation set (0 to 16 mm against 0 to 45mm). This could be mitigated by a k-fold cross validation but we decide to keep working with a hold-out validation set to save a lot of computation time. Nonetheless, in the following, the validation set used to compare models (and not to train them) is the one with the random seed of 43. We make the assumption that this validation set can be interpreted as an intermediary test set, and then implying that evaluation on this set reflects the models' performance on a real inference task.

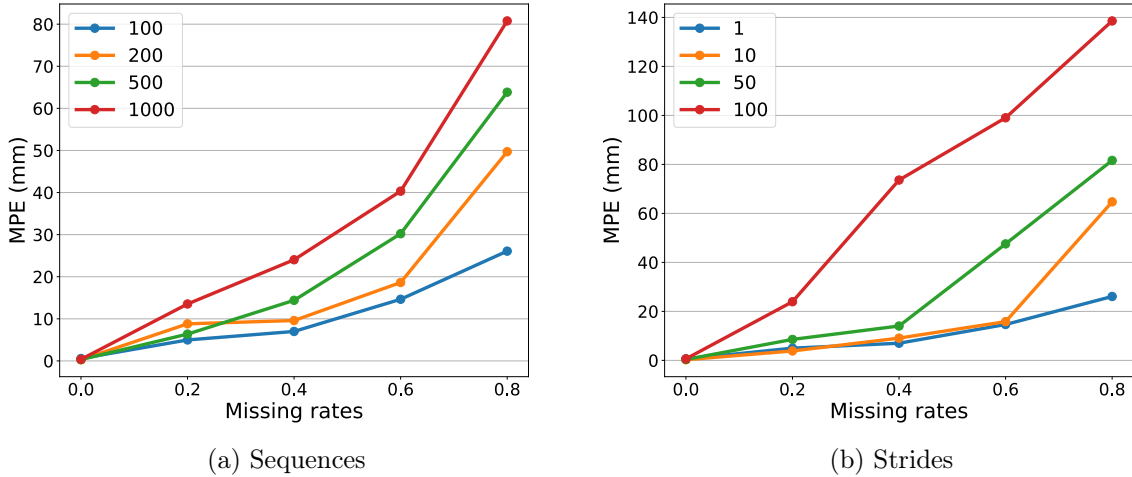


Figure 3.8.: Dataset parameters

Next, we evaluate the influence of the values of T and $stride$. These hyperparameters modify the length of the PyTorch Dataset (in a way explicited in the following python script) so we adapt the training conditions from extreme values to get a fair comparison. For instance, at $stride = 100$, the dataset contains 49 elements ($= T_0/100$) so a batch size $N = 128$ does not make any sense. In that case, we decrease N to 3 to sensibly have the same number of batches as in the case $stride = 1$ and we also increase the number of maximum epochs accordingly. It should also be noted that the sequences length are compared with $stride = 1$, while the different strides are compared with $T = 100$. The figures clearly indicate that we should keep $T = 100$ and $stride = 1$. We do not go below $T = 100$ because of $mask_length = 100$.

3. Methodology

```

1 class MouseTracks(Dataset):
2     """ Mouse Tracking Dataset """
3     def __init__(self, path, seq_length=100, stride=1, mode=0, drop_rate=0, norm=False):
4         ... # initialization, normalization and masking
5
6     def __len__(self):
7         return (len(self.positions_d) - self.seq_length * V) // (self.stride * V) + 1
8         # self.positions_d is the DataFrame directly coming from the original data
9
10    def __getitem__(self, i):
11        ... # slicing and reshaping
12
13    return x, y

```

As a last attempt to improve our single cell LSTM, we introduce some regularization in the form of constraints in the training loss. The first constraint is the BLE but with squared differences in place of absolute values that we call the skeleton loss. The second one also comes BRA [7] and is called the smoothness loss. While the skeleton loss involves the ground truth, the smoothness loss is purely a constraint on the prediction, ensuring the smoothness of the body parts' acceleration. With our notations, the smoothness is calculated in this way:

$$Loss_{sm}(\hat{Y}) = \|O\hat{Y}\|_2^2 \quad \text{with} \quad O = \begin{pmatrix} -1 & 1 & & & & 0 \\ 1 & -2 & 1 & & & \\ & 1 & \ddots & \ddots & & \\ & & \ddots & \ddots & 1 & \\ & & & 1 & -2 & 1 \\ 0 & & & & 1 & -1 \end{pmatrix}_{T \times T}$$

Thus, our training loss takes the form :

$$Loss_{train}(Y, \hat{Y}) = MSE(Y, \hat{Y}) + \alpha Loss_{sk}(Y, \hat{Y}) + \beta Loss_{sm}(\hat{Y})$$

where α and β are two coefficients that we determine through an iterative grid search for the optimum (in the sense of minimisation of MPE). We first find $\alpha = 0.1$ with $\beta = 0$, then we find $\beta = 1e-8$ while fixing $\alpha = 0.1$. We try to find another optimum around $\alpha = 0.1$ with $\beta = 1e-8$ but $\alpha = 0.1$ still gives the best results.

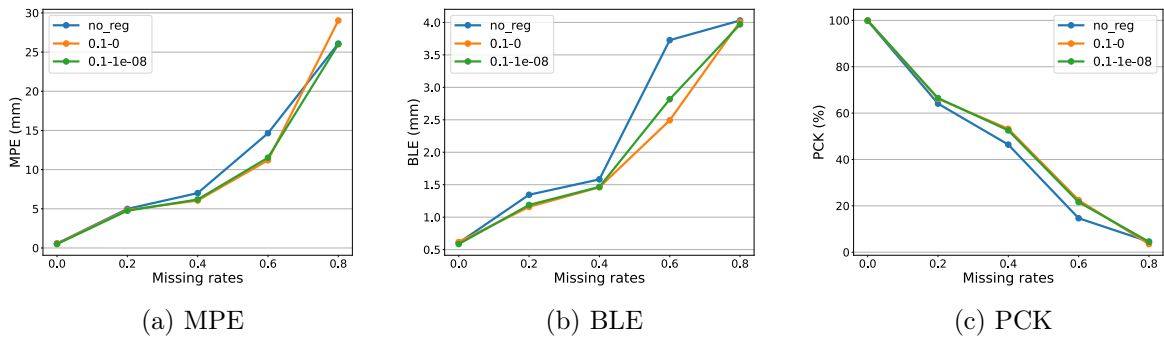


Figure 3.9.: Regularization

The BLE values confirm the effectiveness of the skeleton constraint. Actually, those two figures show that $(\alpha = 0.1, \beta = 1e-8)$ configuration does not really perform better than $(\alpha = 0.1, \beta = 0)$. However, we still retain $(\alpha = 0.1, \beta = 1e-8)$ because it does not induce a deterioration of performance. Quantitatively, the MPE and BLE range of values finally achieved by our

regularized LSTM are acceptable. Although the PCK indicates that the LSTM barely recover any correct parts at a missing rate of 0.8, 30 mm is roughly half of the size of the mouse and is very low compared to $\Delta X, \Delta Y \approx 600$ mm, meaning the reconstructed body parts are in the same area of the overall motion space as the original mouse. Additionally, the maximum BLE value of 4 mm compared to the maximum ground-truth bone lengths (≈ 30 mm) is pretty low.

Working from this regularized LSTM, we try many other configurations like adding layers with dropout or even the BiLSTM but they yield worse metrics. Those observations can be interpreted as increasing the complexity of the model is probably not the good approach to retrieve missing coordinates in our dataset. We decide to keep the single cell regularized LSTM as our final LSTM network that will be evaluated in the last chapter.

ST-GCN

The exploration on LSTM showed that adding information about the mouse structure helps the network to retrieve occluded coordinates more accurately. Therefore, given the graph structure of our mouse model, we investigate ST-GCN. We chose ST-GCN because it is substantially documented and easily accessible. For fairness, experiments are conducted in the same conditions as for LSTM, from the training conditions (maximum number of epochs, EarlyStopping, ...) to the dataset hyperparameters ($N = 128$, $T = 100$, $stride = 1$). ST-GCN takes sequences of shape (N, C, T, V) as input.

We get the whole architecture with the nine blocks of spatial and temporal convolutions and we adapt it to make it output coordinates instead of node features, primarily by deleting the temporal pooling in the layers. We start by training ST-GCN with the three different partitioning strategies presented in the literature review, namely uniform, distance and spatial. We define the head center as the gravity center node.

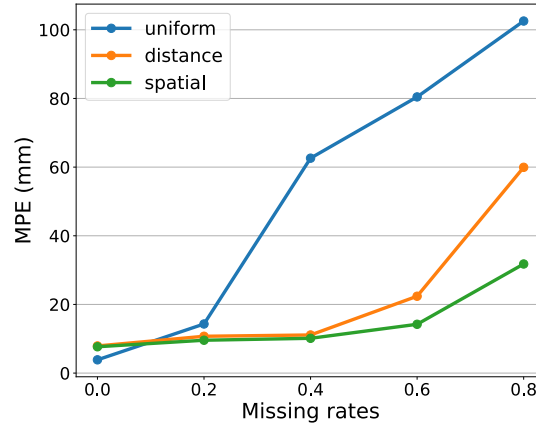


Figure 3.10.: ST-GCN with the full architecture

The spatial configuration partitioning noticeably yields the best MPE so we only retain this one out of those three for the rest of the experimentation. Now, remembering the work of Lino et al. [6] and that LSTM actually performed better with a simple and low-dimensional architecture, we opt to also simplify the ST-GCN architecture. Indeed, the ST-GCN from Lino et al. is made up of only three spatial-temporal blocks and incorporates a partitioning strategy first implemented by Cai et al. [41] that stems from spatial but adds the notion of symmetry. We adopt a ST-GCN composed of three blocks with a maximum embedding dimension twice less than the one of the full ST-GCN. We also adopt the symmetry partitioning strategy and compare it to spatial.

3. Methodology

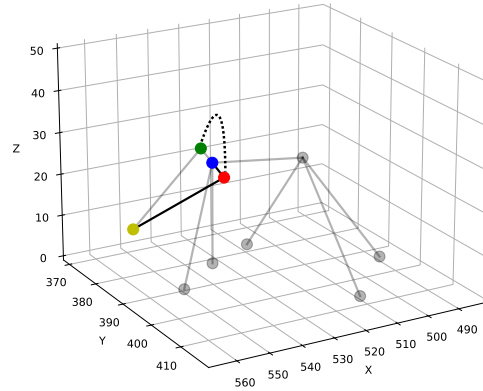


Figure 3.11.: Symmetry Strategy for the left ear

In figure 3.11, the red node (left ear) is the root node and the plain and opaque edges are its spatial edges. The dashed edge is created by the partition strategy. The root node makes its own partition, the blue node is the centripetal node (because head center is the gravity center), the yellow node (nose) is the centrifugal node and the green node (right ear) is the symmetric. This makes a total of four subsets for symmetry compared to three for spatial, meaning more ability to process information in parallel. Furthermore, we assume that the extra edge to the symmetric could really be helpful in the Parts masking.

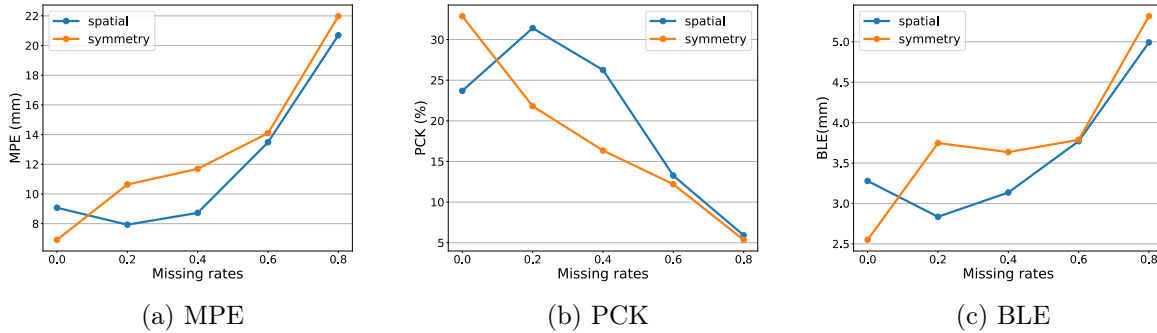


Figure 3.12.: ST-GCN

The results produced by the minimal ST-GCN are slightly better than those of the full architecture (for spatial) while reducing the computation time significantly. Symmetry does not induce an amelioration of the reconstruction, but we choose to not discard it because its performance is not way worse than spatial performance and its implementation concept seems promising for the other types of masking. One thing that strikes on the ST-GCN results is the non-capacity of the networks to learn the identity function, with only a maximum of around 30% for the PCK. Although ST-GCN integrates residual connections, it also comprises many operations such as BatchNorm[52] and sequences of convolutions that hinder the model to approximate the identity. ST-GCN's benefit seems to surface on high sparsity data, as it only performs better than the regularized LSTM at a missing rate of 0.8. Moreover, we would have expected ST-GCN to retain the skeleton structure more precisely but its BLE are also higher than LSTM's ones (but not by a large margin). The models will be extensively and rigorously compared in the last chapter thanks to a procedure that we describe in the following chapter.

4. Evaluation

4.1. Test Sets

The final evaluations of the model are conducted on three different test sets that modify some base parameters of the data generation compared to the original dataset, namely:

1. the Seed test set, where only the seed of the random walk is modified. It isolates pure randomness.
2. the Speed test set, that keeps the default parameters but decreases the base speed of the mouse by half. Like in the original dataset, the mouse increases its speed during motion.
3. and the Angle test set, that also retains all default parameters except for the angle span that is reduced by a factor 5. This means the mouse takes less curved turns during motion.

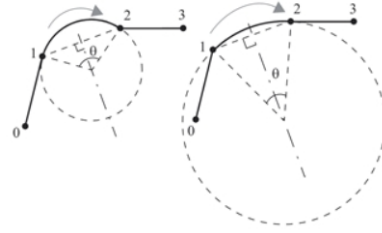


Figure 4.1.: θ is the angle span [53]

We use the same masking function and the same Pytorch Dataset as for the original dataset to load the test sets. After slicing by the masking function, the test sets have $T_0 = 2400$ time steps.

4.2. Baseline

As a baseline, we use a very simple cubic spline interpolation on every coordinate independently. More precisely, this means that for every body part, a cubic spline interpolation is applied on x , another on y and a last one on z .

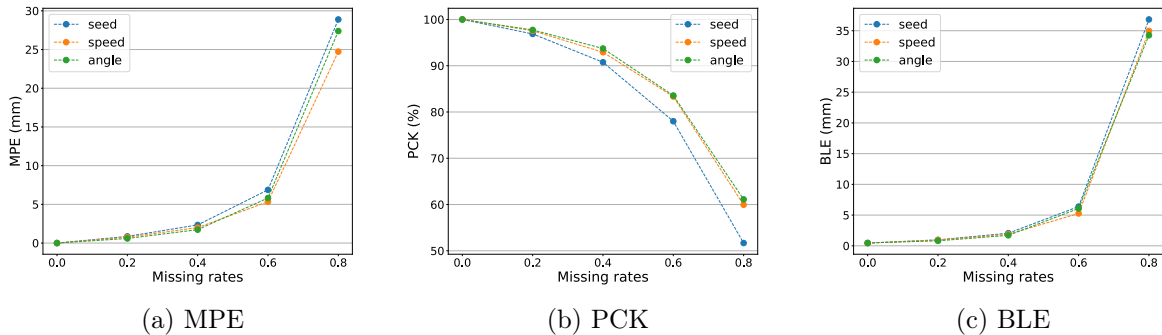
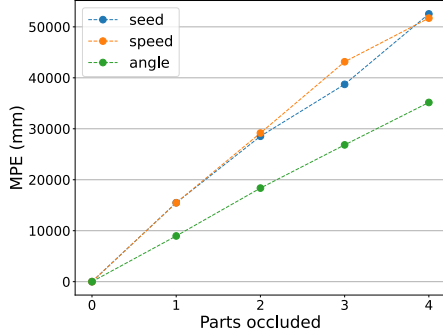


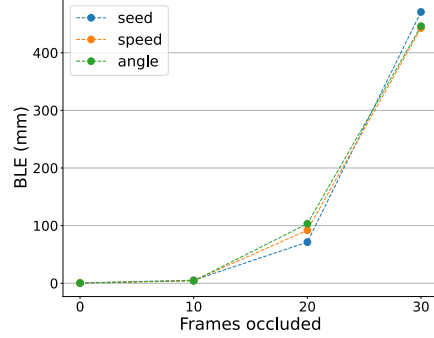
Figure 4.2.: Baseline metrics across the test sets for the Coordinates masking

4. Evaluation

This baseline is adapted to Coordinates but not to the other maskings. In fact, Parts and Frames occlude the same coordinate for a period that is too long for the cubic-spline interpolation to correctly infer the intermediate positions. This results in really high MPE and BLE as shown in Figure 4.3.



(a) MPE for Parts masking



(b) BLE for Frames masking

Figure 4.3.: Baseline metrics across the test sets for the other maskings

Obviously, we cannot compare our results to such absurd metrics, so the spline interpolation baseline is only used in Coordinates. The baseline still gives very high PCK for these masking but only because the spline interpolation retains the non-occluded elements, while our DL models reconstruct everything. For this reason, we also do not use the PCK of the baseline in any masking. For instance, figure 4.4 displays a PCK of approximately 75% for the three test sets. Given that 30 occluded frames (every 100 frames) correspond to 30% of the body parts' positions in the whole dataset, this implies that the spline interpolation actually recovers about 5% of correct body parts in addition to the 70% that were not occluded.

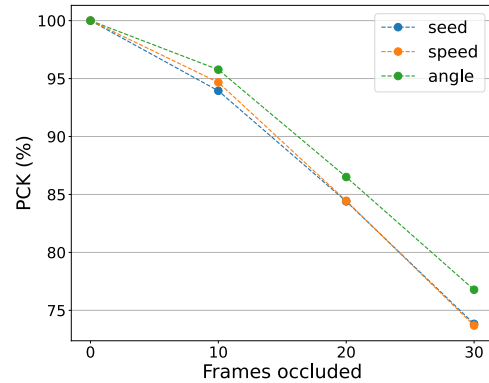


Figure 4.4.: PCK for Frames masking

4.3. Results

To further mitigate the effects of variability and randomness, we respect a strict procedure to quantitatively evaluate and compare our models. We train each model (for every masking and every value of masking) five times on different masking random seeds. These seeds are controlled, meaning they are always the same across the experiments.

```

1  import torch
2  import pandas as pd
3  import numpy as np
4
5  V = 10 # number of body parts
6
7  def masking(data, mask_length, mode, drop_rate, append):
8      masked = np.empty((0, data.shape[1]))
9      data_copy = data.copy(deep=True)
10
11     for i in range(len(data)//(mask_length*V)):
12         rng = np.random.default_rng(seed=42+append+i)
13         ...

```

In practice, we introduce an integer variable, *append*, that takes values from 1 to 5 and is added to the original seed as depicted in the Python code above. Additionally, we use *append* on the seed for the training-validation split of the dataset, meaning we get five different masking, each one with its own train-validation split. We keep the seed of the test sets constant during the evaluation. Furthermore, we replace ST-GCN by AGCN (2s-AGCN[42] without the double stream) with exactly the same number of blocks and adjustments we provided for ST-GCN in the methodology section. The adaptive graph convolution is well suited for handling missing data and we also observe better quantitative results on the test sets compared with ST-GCN. All the figures in this section present the mean value of the metric across the five runs, with error bars representing the standard-deviation of the metric. We present and compare models performance for every type of masking.

4.3.1. Coordinates

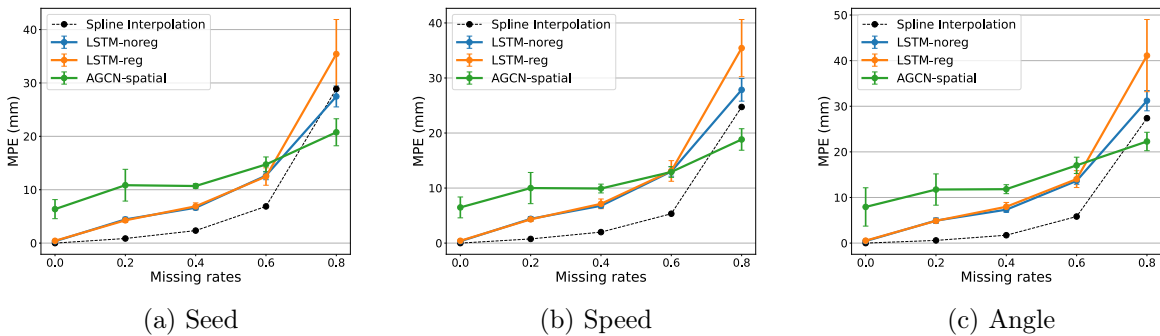


Figure 4.5.: MPE

4. Evaluation

Among the GCN methods, AGCN-spatial (AGCN with a spatial configuration partition strategy) yields the best results on Coordinates, that is why we only display this one along with the LSTM and the cubic spline interpolation. As expected, the MPE of the networks have a bigger range of values in comparison with the Methodology chapter 3. Nonetheless, they remain within a reasonable range, with a maximum of 35 mm for Seed and Speed and 40 mm for Angle. Our regularized LSTM appears to struggle at high sparsity of data, as its wide bar errors show. This is surprising in the sense where we would expect the constrained LSTM to be more stable but, as a matter of fact, LSTM without regularization performs significantly better and exhibits greater stability at a missing rate of 0.8 and this across the three datasets. One thing that could explain the lower performance of the regularized LSTM is that the coefficients α and β have been optimized on the original dataset and their optimal values could be really different for every of the other three test sets. Furthermore, the instability of the regularized LSTM at high sparsity is probably a consequence of the constraint themselves, notably the smoothness loss that only depends on the reconstructed data. This implies that this constraint also heavily depends on the occlusion, since networks reconstruct data from its corrupted version. Therefore, different masking seeds lead to very dissimilar smoothness values (at least in the beginning of the training) and ultimately to dissimilar performance on the same test set. This effect is all the more noticeable on Angle because the smoothness loss constrains acceleration and acceleration in Angle is very different from the one in the original dataset (because the turns have different intensities). Thus the optimal α and β for Angle must also be really different from the one we found.

Figure 4.6b seems to corroborate these affirmations. The bone-length regularization does not seem to work anymore but its BLE is pretty much uniform over the different seeds. By elimination, the variability observed in the MPE must be due to the smoothness loss. More generally, LSTM perform very similarly for other missing rates across the test sets while following the spline interpolation curve. Given the ability of LSTM to map the identity function, it is plausible that they actually carry out some kind of interpolation, monitored by the reconstruction loss. AGCN has a fairly different behavior against sparsity. Its curve is way smoother and it is the only network to constantly outperform the cubic-spline interpolation at a missing rate of 0.8 on the three test sets, demonstrating its robustness to missing data. However, this good performance still translates to a very low PCK, which is expected with that amount of occlusion. In fact, we cannot really draw conclusions from the results observed on Coordinates but it allowed to put our models in perspective with a cubic-spline interpolation. To effectively discriminate the models, we leverage the two others masking: Parts and Frames.

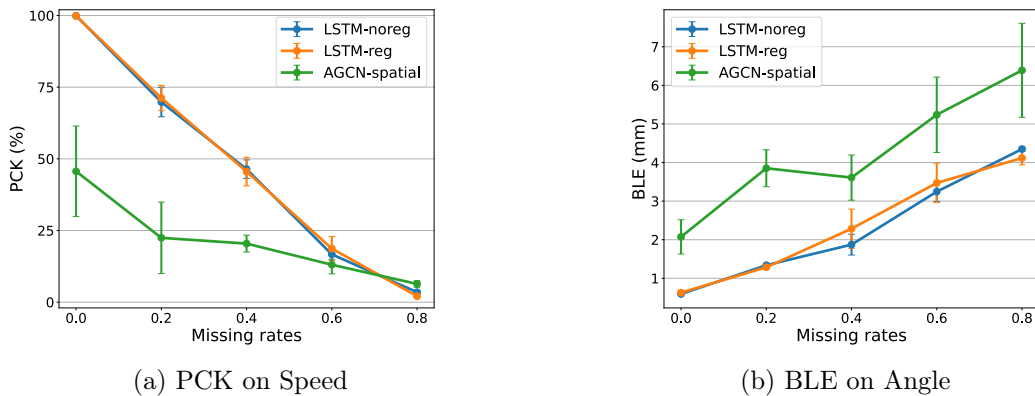


Figure 4.6.: Other metrics

4.3.2. Parts

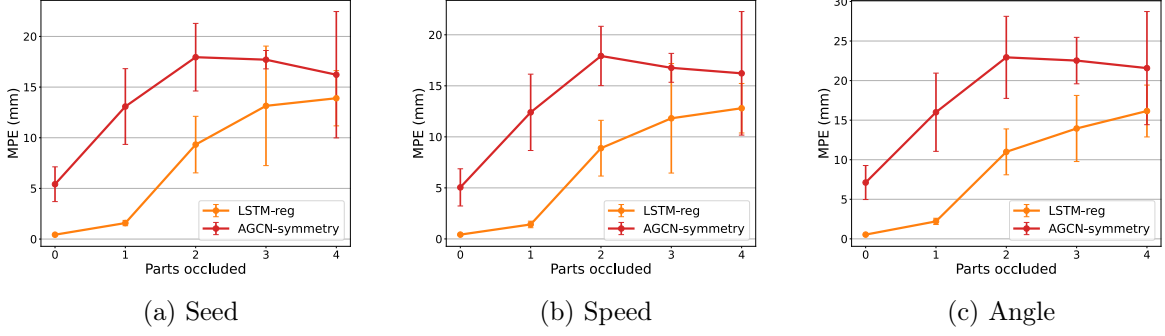


Figure 4.7.: MPE

On Parts, AGCN with the symmetry partitioning strategy yields better results than with the spatial one but not by a significant margin. An increase of performance for the symmetry strategy was expected, as it offers more flexibility to the graph by adding supplementary edges. Nonetheless, AGCN is an adaptive model so we can imagine that it creates symmetric connections in the spatial configuration as well. For LSTM the regularized version achieves higher performance. Thereby, we only compare these two models. The most striking elements in this figures are the very wide error bars for both of the models, highlighting the instability of the models on Parts. To explain where this variability comes from, we analyze results from Coordinates. Figure 4.8 reveals that, under uniform high-rate occlusion (Coordinates), both LSTM and AGCN have more difficulty to recover the position of the nose (0) and AGCN also have trouble with the tail tip (7). Positions of the joints (head center (8) and tail base (9)) seem to be a bit easier to retrieve for AGCN.

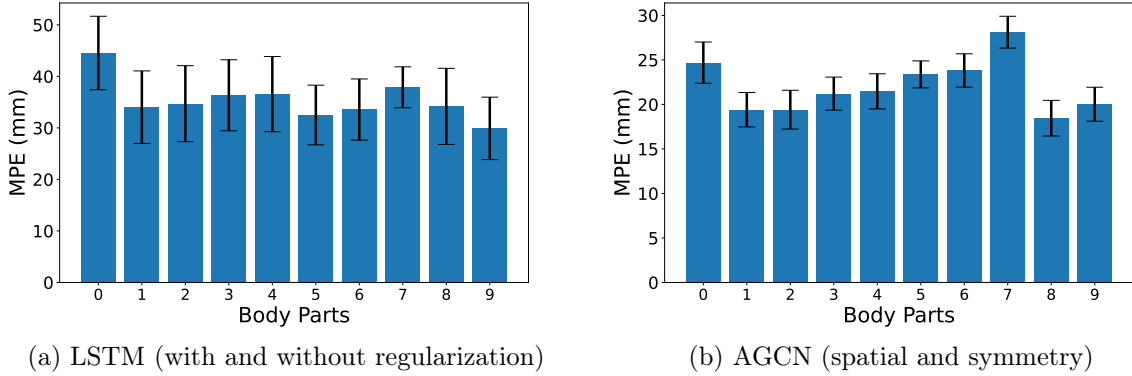


Figure 4.8.: MPE per body part on Seed for a missing rate of 0.8

LSTM and AGCN have trouble with the nose positions probably because of the particular movement of this body part, that rotates relatively to the mouse's body. It is a bit more precise on the joints positions because they have simpler trajectories, defining the mouse motion in the overall space. We can try additional explanation for AGCN. Since AGCN leverages the connectivity of the nodes, a node with many neighbors can aggregate more information. The head center relies on the left and right ears, the left and front paws and the tail base; while the tail tip only relies on the tail base. This would explain the significant difference in MPE. However, this reasoning is not totally true with AGCN because it is supposed to be adaptive, meaning AGCN can create additional edges according to the situation.

4. Evaluation

In any case, Figure 4.8 still shows that our models do not retrieve the parts with the same accuracy, and therefore masking different body parts can yield very different results. Our models would certainly give better results in a situation where only the head center is masked compared to a situation where only the nose is masked for LSTM or the tail tip for AGCN.

Having addressed the instability of the masking, we can now proceed to the quantitative analysis of the results. LSTM consistently outperforms AGCN, even if their performance seem to converge when the number of parts occluded increases.

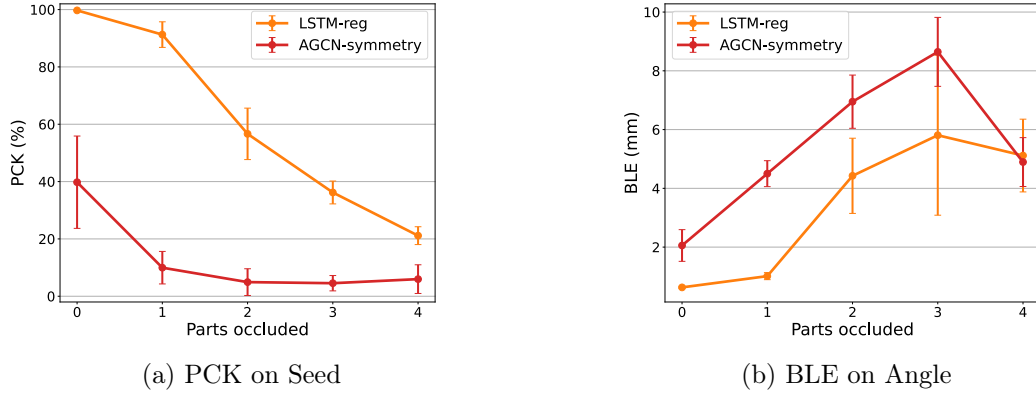
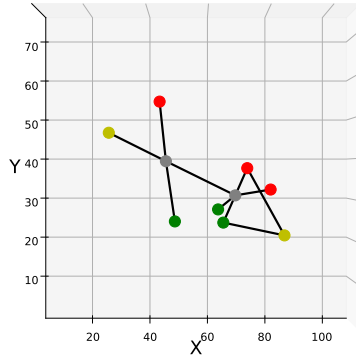
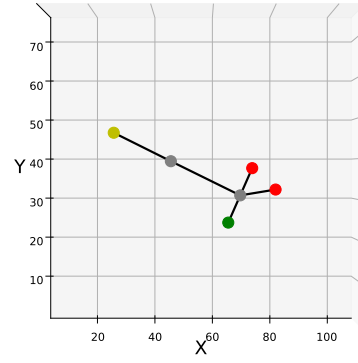


Figure 4.9.: Other metrics

This gap in the performance is confirmed by Figure 4.9a where LSTM can predict about twice more correct positions than AGCN for 4 occluded parts. Additionally, we notice again a higher MPE on Angle like in Coordinates. AGCN suffers from a considerable decline in performance here, whereas it remained stable across the three datasets for Coordinates. However, it makes sense for the models to have more difficulty to infer missing parts in Angle because the range of skeleton poses is modified compared to the training setting. In contrast to Coordinates, we can visualize some skeleton reconstructions by our models. These visualizations do not permit definitive conclusions about model performance, but they provide insight into the reliability of the metrics.



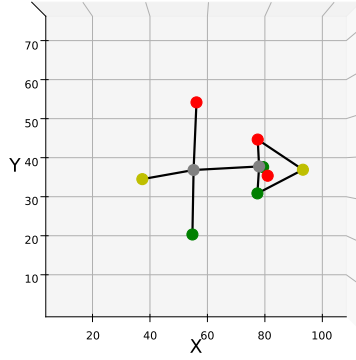
(a) Ground-Truth Skeleton



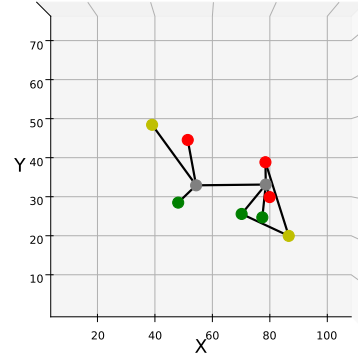
(b) Masked Skeleton

Z-diff: 0.13mm

Z-diff: 0.23mm



(c) LSTM-reg

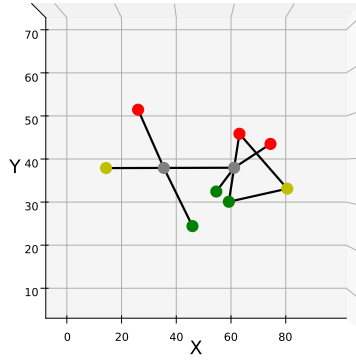


(d) AGCN-symmetry

Figure 4.10.: Example of a skeleton reconstruction for 4 occluded parts on Seed. Z-diff is the mean difference between predicted and ground-truth Z coordinates.

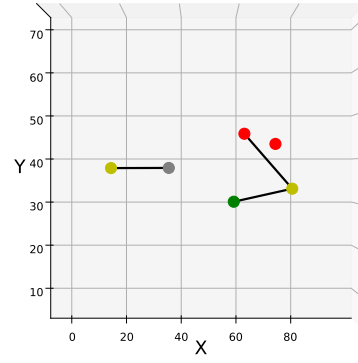
We visualize a frame where AGCN performs well with a low MPE (7.7 mm) in Figure 4.10. Both models exhibit a very low Z-difference, as expected given that Z remains uniform throughout the motion for all body parts. Moreover, both of them present recognizable skeleton but with the left front paw on the right side. Actually, the left front paw prediction is quite precise but it is the head center that is inaccurately localized for the two methods, even though neither of them is masked. AGCN skeleton looks distorted compared to LSTM skeleton that seems more structured. Another notable discrepancy in the predictions of the two methods on this frame concerns the central axis of the mouse, namely the nose, the head center, the tail base and the tail tip. In fact, the head rotates so the nose is rarely in the axis, as in this frame where the head is tilted a bit on the right side of the mouse. AGCN recovers the position of the nose very accurately and the tail tip is a bit off on the X axis. LSTM wrongly predicts the positions of the nose and the tail tip but keeps them aligned on the central axis while displaying a more usual head shape than AGCN. The position of the hind paws also look more natural with LSTM because they respect the symmetry centered on the tail base. Overall, the good-looking structure of the LSTM skeleton translates to a lower BLE (2.4 mm) compared to the AGCN skeleton (3.3 mm). Although LSTM MPE (11.7 mm) is higher by a meaningful margin, visualization shows that its predicted skeleton is somewhat more accurate than the skeleton predicted by AGCN.

4. Evaluation



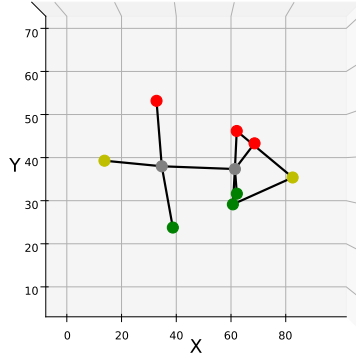
(a) Ground-Truth Skeleton

Z-diff: 0.01mm

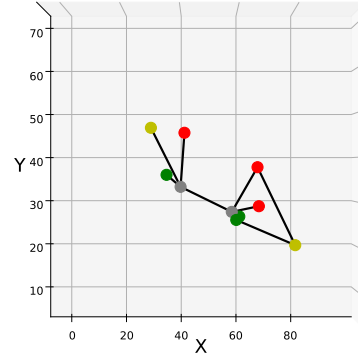


(b) Masked Skeleton

Z-diff: 0.25mm



(c) LSTM-reg



(d) AGCN-symmetry

Figure 4.11.: Example of a skeleton reconstruction for 4 occluded parts on Angle

Next, we display a frame where the MPE of LSTM is very low (3.6 mm) in Figure 4.11. Here, LSTM noticeably produces a better skeleton than AGCN. LSTM reconstructs non-occluded body parts with a very high precision. The left front paw and the nose are slightly misaligned, with the paw positioned nearer to the main body and the nose drawn toward the central axis as in figure 4.10c. LSTM also accurately retrieves the location of the head center, and the hind paws with a little offset but it is still retaining the central symmetry for these parts. However, the absence of symmetry in the front paws renders the right front paw the most evident misprediction, despite the ground-truth location being relatively close. Conversely, AGCN produces a skeleton of barely discernable structure. As in 4.10d, the hind paws are too close to the central axis and the left hind paw and the tail roughly forms the same angle. In addition, the head structure is irregular. While AGCN attains the same MPE on this frame as the LSTM in figure 4.10 (11.7 mm), the quality of the reconstructed skeleton differs substantially. The BLE effectively represents this aspect (3.9 mm for AGCN and 1.3 mm for LSTM here). Overall, LSTM skeleton reconstruction on those two frames is considerably more precise than AGCN one. Moreover, the maximum MPE and BLE are “only” of 11.7 mm and 3.9 mm respectively for AGCN in Figure 4.11. Figure 4.7 shows that average MPE over frames is about 23 mm and the BLE is around 5 mm for AGCN where four body parts are occluded. In those conditions, we can almost confidently affirm that AGCN does not reconstruct the mouse skeleton across the motion.

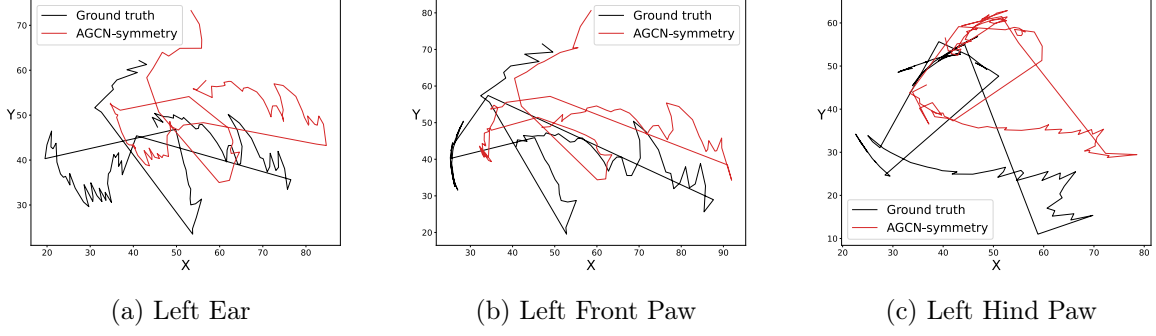


Figure 4.12.: Tracking of reconstructed body parts for AGCN on Seed

To better understand how LSTM achieves such strong performance, we do three additional experiments where some parts are occluded during the whole motion while all the other parts stay visible. In the first one, the three left body parts are occluded, in the second one we mask the front paws and in the last one we mask the hind paws. Figure 4.12 and Figure 4.13 provide the tracking of the reconstructed body parts over one sequence for the first masking. The results are unequivocal, LSTM maps the the ground-truth track with a very high accuracy. AGCN tracking can seem really bad at first, but on a second sight we notice the same shapes in the prediction than in the ground truth. In Figure 4.10 and Figure 4.11, we observed that the hind paws are offset in a similar manner and Figure 4.12 confirms that this offset is not specific to the frames visualized. This appears to be an intrinsic limitation of our model, which could potentially be addressed through further tuning. However, the track visualization still confirms that AGCN can learn temporal dependencies.

Since the LSTM produced results of surprisingly high accuracy, we may believe that LSTM actually learns every part motion independently. In order to verify this possibility, we feed the LSTM with the front paws positions only and totally mask them during the whole motion. In this case, LSTM cannot follow the tracks ($MPE \approx 200$ mm) so this proves that LSTM does not only reconstruct tracks in parallel, but leverages the structure of the mouse instead.

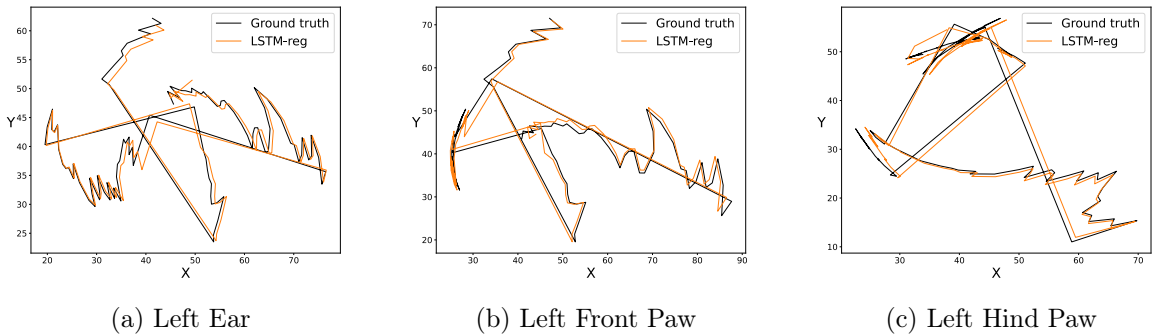


Figure 4.13.: Tracking of reconstructed body parts for LSTM on seed

4. Evaluation

4.3.3. Frames

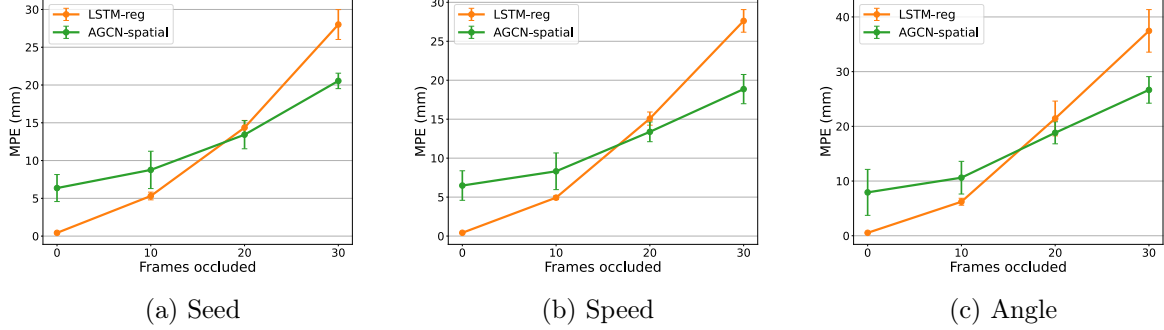


Figure 4.14.: MPE

Frames is expected to be the most challenging masking for the networks. However, the MPE observed are still in a reasonable range even for Angle. As for the other maskings, MPE is higher on Angle than on the other tests sets, whereas we could have expected an increased MPE on Speed as well. Indeed, without any landmark (the whole skeleton is masked), the models should predict next positions according to the pace they were trained on. This would result in a significant offset at the end of the occlusion period. AGCN is less impacted by the increase in the number of occluded frames and achieves a considerably lower MPE at 30 occluded frames. Nonetheless, PCK and BLE (Figure 4.15) indicates that AGCN may not perform better than LSTM. LSTM presents superiors results in those two metrics LSTM for any number of occluded frames and by a significant margin for the PCK. Visualizing the reconstructions of the skeleton by the methods during an occlusion episode offers greater insight into the models' performance and capacities.

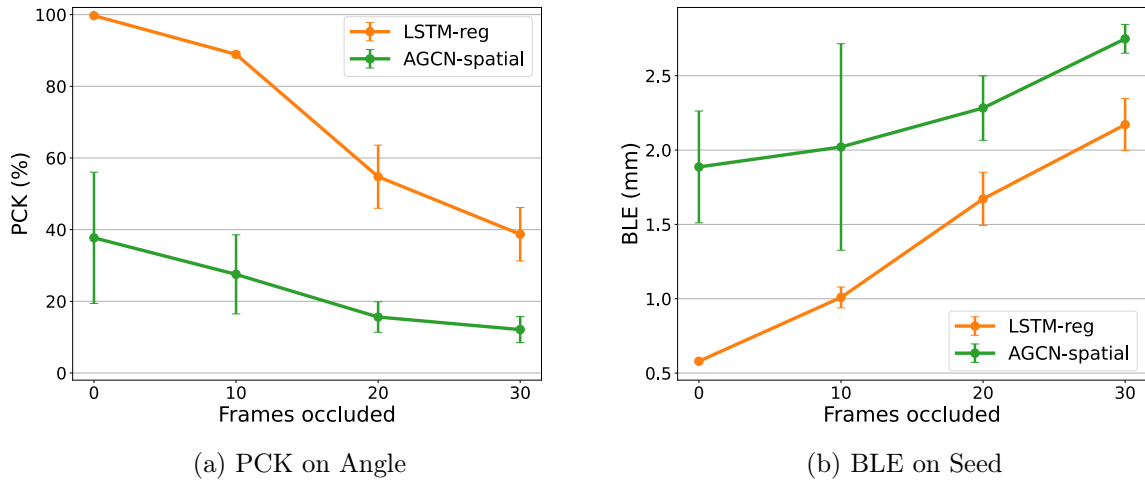
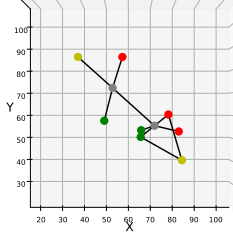
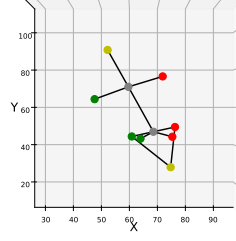


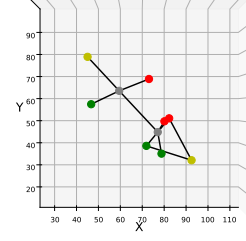
Figure 4.15.: Other metrics



(a) 1st Frame



(b) 5th Frame



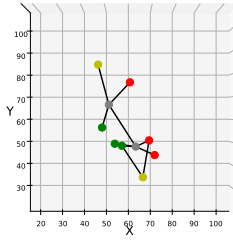
(c) 10th Frame

Ground Truth

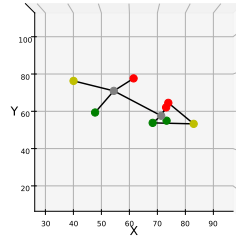
Z-diff: 0.11mm

Z-diff: 0.13mm

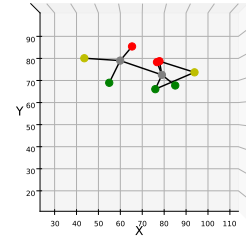
Z-diff: 0.05mm



(d) 1st Frame



(e) 5th Frame



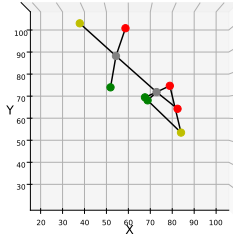
(f) 10th Frame

LSTM

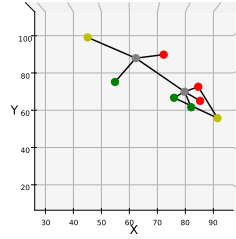
Z-diff: 0.09mm

Z-diff: 0.06mm

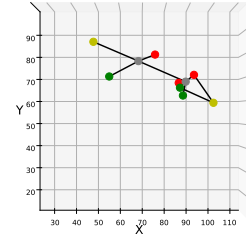
Z-diff: 0.13mm



(g) 1st Frame



(h) 5th Frame



(i) 10th Frame

AGCN-spatial

Figure 4.16.: Example of skeleton reconstructions during the occlusion of 10 frames on Speed

We display an interesting example where LSTM and AGCN outputs interpretable skeletons on Speed. Across the six reconstructions, only the 10th Frame skeleton of AGCN has a remarkable mislocated body part relatively to the skeleton body (the left front paw on the right side). The reconstructed skeletons remain notably consistent throughout the occluded period, but the more striking feature of these figures is the change in skeleton orientation. On the first frame, AGCN effectively reproduces the orientation of the ground truth while LSTM is a bit misaligned. Then, both reconstructed skeleton start a rotational movement drawing away from the ground truth. LSTM rotation is the most abrupt. It seems like they identified the previous motion of the ground truth as the beginning of a rotation, which might actually be the case but we cannot see it on the 10th frame of the ground truth. This rotation of the reconstructed skeletons in the occluded conditions lead to a considerable offset with the ground truth. We can emphasize on the

4. Evaluation

fast rotation of the LSTM here. We know that LSTM retains strong temporal cues and in Speed the mouse is actually twice slower. By extrapolation, we can imagine that being in a totally occluded situation, LSTM predict positions according to the pace it learned. If it discerned a rotational movement before the occlusion, LSTM would predict a faster rotation than the Speed ground truth, hence the observations.

Prior to concluding our assessment of the models' performance, we provide observations regarding the test sets. The stable performance between Seed and Speed across the masking modalities seems to imply that the models are not disturbed by a decrease of pace. On the contrary, they overall achieve slightly better MPE on Speed compared to Seed. A decrease of pace can also be seen as an increase of "spatial resolution" as every movement is described by more frames. Angle is the most challenging test set for our models, as evidenced by the consistently and significantly higher MPE relative to the other test sets. In fact, this gives substantial information about our methods. Angle reduces the angle span, so its range of poses should be strictly included in the range of poses of the original dataset. However, we observe a deterioration of performance on this smaller range. A conclusion we may draw is that a network intended for accurate skeleton reconstruction should be trained on a range of poses comparable to those it is expected to reconstruct. This may seem trivial but one could expect a network trained on a very large number of poses to demonstrate good performance on a smaller amount of poses but here this is not the case for our models.

The experiments we realized all point in the direction that LSTM outperforms AGCN. LSTM displayed outstanding results on Parts particularly. Nonetheless, these good results compared to AGCN should be placed in proper perspective. The LSTM was the first network we implemented and also the one for which hyperparameters such as the sequence length and the stride of the sequences were optimized. When training and testing AGCN with the LSTM conditions, the former still proved to be efficient in some situations. Further tuning of this network could probably yield better results. Besides, AGCN may be limited by the mouse's design. In HAR, skeleton typically comprises about 20 or more joint which logically provides more information.

5. Conclusion

In this work, we investigated the skeletal motion data reconstruction of a simulated mouse in various occlusion scenarios. We chose two DL networks, namely LSTM and AGCN, to perform this task after the study and analysis of existing methods manipulating skeletal data. Our results suggest that LSTM would be more suited for the task of skeleton reconstruction but this interpretation can be discussed. The first step was to think of and implement artificial occlusions on the simulated dataset that could produce valuable output after exploitation by our DL networks. With this objective in mind, three different masking strategies were adopted. Firstly, the Coordinates masking that is used for a quantitative comparison purpose. It considers the entire sequences of positions and occlude individual coordinates. The two other masking strategies can be seen as a spatial and a temporal masking. The Parts masking (spatial) occludes up to four body parts for 3 s, and the Frames (temporal) masking occlude contiguous series of frames for durations going from 1/3 to 1 s.

Finding a reliable method that reconstructs skeletal data as an end task was a first hurdle. LSTM was selected for its very well-known ability to handle sequential data and served as a first method to explore the dataset and its characteristics. Given the strong performance of the single-cell LSTM, we attempted to enhance the architecture by adding layers and adopting a BiLSTM but the results were considerably worse. We also tried to implement BRA [7] without any success. Thereafter, ST-GCN that is a method for skeleton-based HAR was adapted in structure to output positions over time with metrics comparable to the single-cell LSTM. We subsequently replaced ST-GCN with AGCN after observing that AGCN yielded better results on the test sets. The idea behind selecting LSTM and AGCN is to compare a method emphasizing on the temporal dependencies, with a method integrating the skeleton structure in its framework and thereby putting more accent on the spatial relationships. Ultimately, LSTM came across as the most robust method to reconstruct the skeletal motion data in our dataset. Nevertheless, the selection of hyperparameters common to the two networks was done relatively to LSTM performance, which put comparison results promoting LSTM into perspective.

Moreover, it is unlikely that a simple LSTM would have such good results when modeling the movements of a real mouse. As mentioned throughout the thesis, our simulated dataset is of limited size but the motion generated is also pretty simple. To begin with, there is no displacement along the Z axis and the paws move synchronically in repeated movements. Therefore, its succession of poses is highly correlated in the temporal domain, even if the whole motion of the mouse is random. LSTM that is designed to capture sequential dependencies has no trouble to recover the sequences, on condition that the occlusion rate is not too high. AGCN struggles with the tracking of the body parts' positions over time because it is originally more of a pose-based method. Learning complex temporal relationships is not its primary function.

On the contrary, Spatio-Temporal Transformers can effectively capture complex temporal relationships together with the spatial relationships. A recent paper, Masked Autoencoder for Transformer-based Estimation and Reconstruction (MATER) [54], adopts the same approach as SkeltonMAE [3] but for mice instead of humans. The authors leverage a Spatio-Temporal Transformer in their autoencoder to reconstruct mice skeletal motion data in extreme masking conditions. The trained encoder is then used together with a projection head for behavior classification. They report that their autoencoder can reconstruct mouse poses even under severe

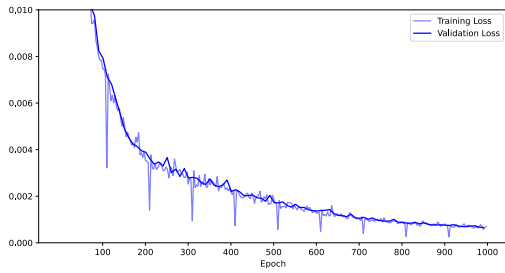
5. Conclusion

occlusion, achieving a PCK of 83 % with 90 % of the input data masked. Although they do not specify the PCK threshold used, the performance remains impressive.

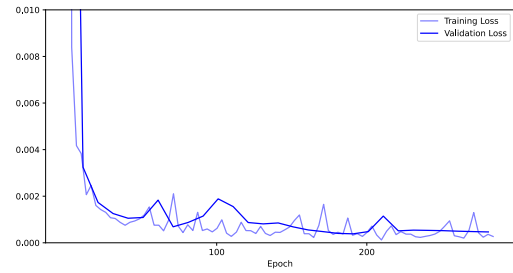
Finally, although we downplayed the LSTM results, we should not overlook its consistency and strong performance. In the introduction, we made the decision to investigate skeleton-based methods but LSTM performance suggests that exploiting the skeletal motion data as a multivariate time series could prove to be efficient. The experimentation of total occlusion of the left side demonstrated the LSTM ability to make correlations between the body parts. We can hypothesize that more sophisticated methods considering multivariate time series as input modalities could capture these correlations in more complex settings. Consequently, applying forecasting models such as CSDI [10] SAITS [11] and Informer [12] to skeletal data reconstruction represents a promising direction for future work.

A. Annex

A.1. Training



(a) LSTM



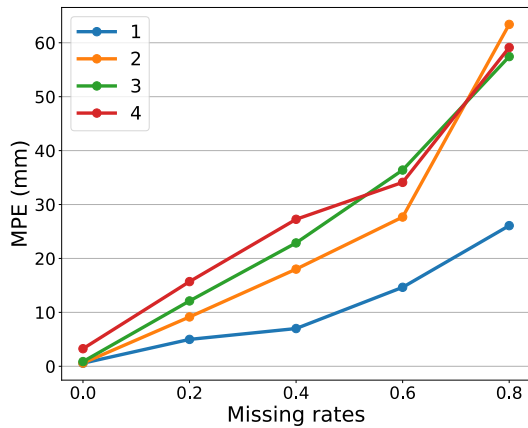
(b) AGCN

Figure A.1.: Typical training curves

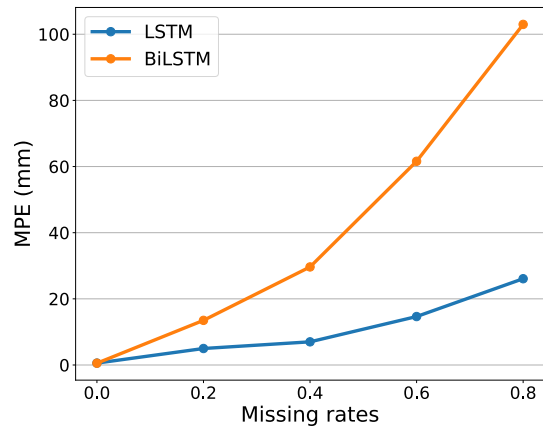
AGCN networks typically converge between 300 and 800 epochs for every masking and rate of occlusion. LSTM converge more slowly and could still be optimized after 1000 epochs.

A.2. LSTM Exploration

We present some additional curves on Coordinates.



(a) Layers



(b) BiLSTM

Figure A.2.: MPE

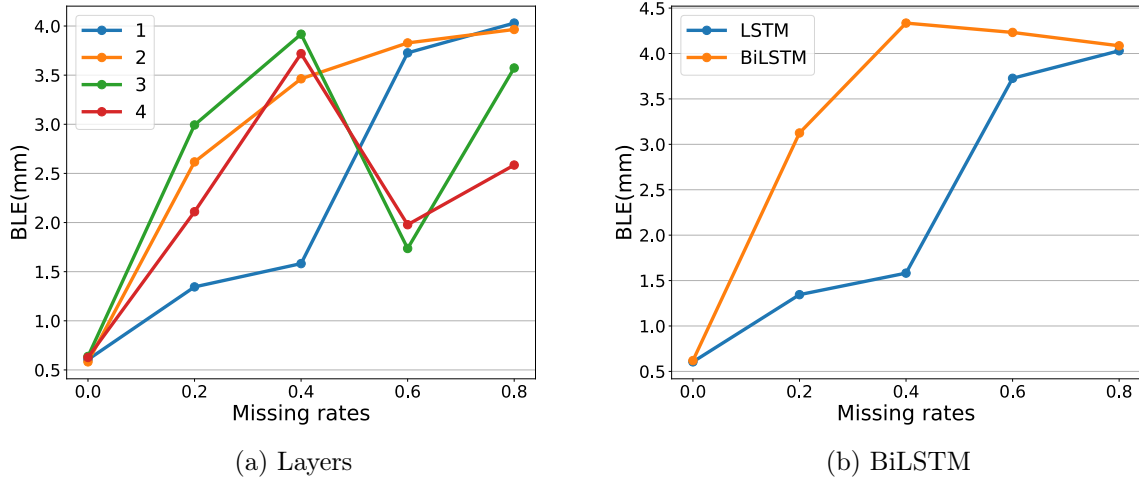


Figure A.3.: BLE

Even with very high MPE, BLE never explode. This means that the predicted body parts stay in the same neighborhood.

A.3. BRA

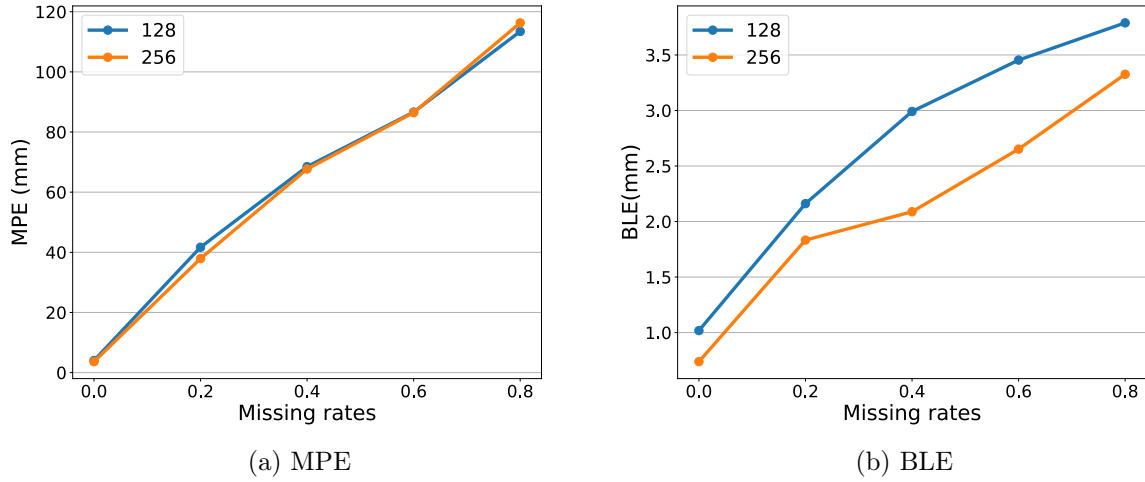


Figure A.4.: BRA [7]

The BRA architecture was adopted as the evolution of the LSTM but we could not get it to output satisfactory results, even after a full tuning of the original architecture.

A.4. Data Augmentation

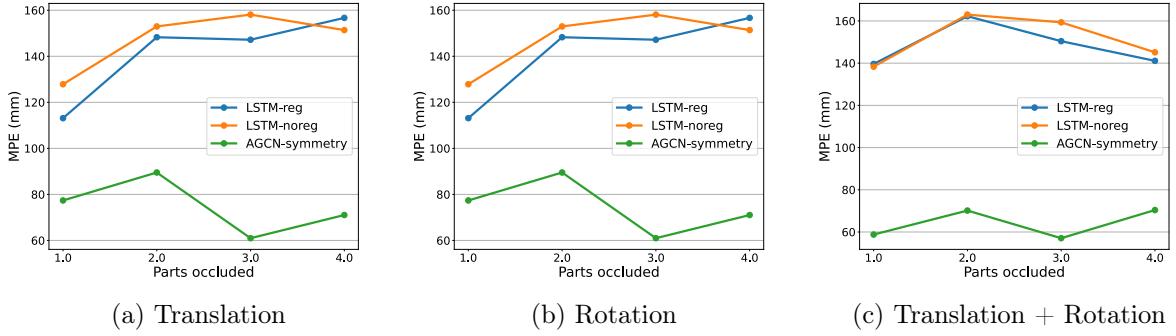


Figure A.5.: Data augmentation for Parts tested on Angle

We compute two kinds of augmentation that double the size of the dataset. The first one is a translation of 200 mm on the three coordinates and the second one is a rotation of 90° relative to Z. The network are trained with the same hyperparameters as in the Results chapter 4. The translation improves AGCN MPE but not as much as it improves LSTM MPE. These results should be put in relation with Figure A.1a. AGCN do not take fully advantage of the augmentation because it converges way faster than LSTM. The translation is equivalent to a run on 2000 epochs for the LSTM because the temporal relationships are the same in the augmented data. However, the rotation causes the MPE to go to very high values. Here again, LSTM runs for 1000 epoch and not AGCN so that may be why AGCN actually present lower MPE, even if the value is objectively very high. The combination of the two augmentations triples the original size of the dataset and yield results similar to the rotation only.

Bibliography

- [1] Olaf Hellwich et al. *Tracking Mouse from Incomplete Body-Part Observations and Deep-Learned Deformable-Mouse Model Motion-Track Constraint for Behavior Analysis*. Comment: 10 pages. Jan. 2025. arXiv: 2501.11030 [cs].
- [2] Marcus N. Boon et al. “Mechanical Problem Solving in Mice”. In: *bioRxiv : the preprint server for biology* (2025). eprint: <https://www.biorxiv.org/content/early/2025/08/25/2024.07.29.605658.full.pdf>.
- [3] Wenhan Wu et al. *SkeletonMAE: Spatial-Temporal Masked Autoencoders for Self-supervised Skeleton Action Recognition*. Comment: ICME 2023 Workshop. May 2023. arXiv: 2209.02399 [cs].
- [4] Yuhan Zhang et al. “STST: Spatial-Temporal Specialized Transformer for Skeleton-based Action Recognition”. In: *Proceedings of the 29th ACM International Conference on Multimedia*. Virtual Event China: ACM, Oct. 2021, pp. 3229–3237.
- [5] Wonseok Lee, Seonghee Park, and Taejoon Kim. “Denoising Graph Autoencoder for Missing Human Joints Reconstruction”. In: *IEEE Access* 12 (2024), pp. 57381–57389.
- [6] Filipa Lino, Carlos Santiago, and Manuel Marques. “3D Human Pose Estimation with Occlusions: Introducing BlendMimic3D Dataset and GCN Refinement”. In: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. Seattle, WA, USA: IEEE, June 2024, pp. 4646–4656.
- [7] Shujie Li et al. “Bidirectional Recurrent Autoencoder for 3D Skeleton Motion Data Refinement”. In: *Computers & Graphics* 81 (June 2019), pp. 92–103.
- [8] Lu Ji et al. “Missing Data Recovery for Human Mocap Data Based on A-LSTM and LS Constraint”. In: *2020 IEEE 5th International Conference on Signal and Image Processing (ICSIP)*. Nanjing, China: IEEE, Oct. 2020, pp. 729–734.
- [9] Jun Liu et al. *Spatio-Temporal LSTM with Trust Gates for 3D Human Action Recognition*. July 2016. arXiv: 1607.07043 [cs].
- [10] Yusuke Tashiro et al. “Csdi: Conditional score-based diffusion models for probabilistic time series imputation”. In: *Advances in neural information processing systems* 34 (2021), pp. 24804–24816.
- [11] Wenjie Du, David Côté, and Yan Liu. “SAITS: Self-attention-based Imputation for Time Series”. In: *Expert Systems with Applications* 219 (2023), p. 119619.
- [12] *Multivariate Probabilistic Time Series Forecasting with Informer*. Accessed: 2025-09-09. URL: <https://huggingface.co/blog/informer>.
- [13] Ngoc Nguyen et al. “Applying Deep Learning Models to Action Recognition of Swimming Mice with the Scarcity of Training Data.” in: *Proceedings of the 12th International Joint Conference on Biomedical Engineering Systems and Technologies*. Prague, Czech Republic: SCITEPRESS - Science and Technology Publications, 2019, pp. 270–275.
- [14] Amir Shahroudy et al. “NTU RGB+D: A Large Scale Dataset for 3D Human Activity Analysis”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, June 2016, pp. 1010–1019.

- [15] Jun Liu et al. “NTU RGB+D 120: A Large-Scale Benchmark for 3D Human Activity Understanding”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.10 (Oct. 2020). Comment: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), pp. 2684–2701. arXiv: 1905.04757 [cs].
- [16] Will Kay et al. *The Kinetics Human Action Video Dataset*. May 2017. arXiv: 1705.06950 [cs].
- [17] Catalin Ionescu et al. “Human3.6M: Large Scale Datasets and Predictive Methods for 3D Human Sensing in Natural Environments”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.7 (July 2014), pp. 1325–1339.
- [18] Jennifer J Sun et al. “The Multi-Agent Behavior Dataset: Mouse Dyadic Social Interactions”. In: (2024).
- [19] Cristina Segalin et al. “The Mouse Action Recognition System (MARS) Software Pipeline for Automated Analysis of Social Behaviors in Mice”. In: *eLife* 10 (Nov. 2021), e63720.
- [20] Christian Szegedy et al. *Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning*. 2016. arXiv: 1602.07261 [cs.CV].
- [21] Alejandro Newell, Kaiyu Yang, and Jia Deng. *Stacked Hourglass Networks for Human Pose Estimation*. 2016. arXiv: 1603.06937 [cs.CV].
- [22] Patrik Reiske et al. *Mouse Lockbox Dataset: Behavior Recognition for Mice Solving Lockboxes*. Comment: Accepted and published (poster) at the CV4Animals: Computer Vision for Animal Behavior Tracking and Modeling workshop, in conjunction with Computer Vision and Pattern Recognition (CVPR) 2025. June 2025. arXiv: 2505.15408 [cs].
- [23] Alexander Mathis et al. “DeepLabCut: Markerless Pose Estimation of User-Defined Body Parts with Deep Learning”. In: *Nature Neuroscience* 21.9 (Sept. 2018), pp. 1281–1289.
- [24] Cailing Wang and Jingjing Yan. “A Comprehensive Survey of RGB-Based and Skeleton-Based Human Action Recognition”. In: *IEEE Access* 11 (2023), pp. 53880–53898.
- [25] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (Nov. 1997), pp. 1735–1780.
- [26] Haşim Sak, Andrew Senior, and Françoise Beaufays. *Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition*. Feb. 2014. arXiv: 1402.1128 [cs].
- [27] *PyTorch LSTM*. Accessed: 2025-09-09. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html#lstm>.
- [28] *LSTM architecture*. Accessed: 2025-09-09. URL: https://en.wikipedia.org/wiki/Long-short-term_memory#/media/File:LSTM_Cell.svg.
- [29] Greg Van Houdt, Carlos Mosquera, and Gonzalo Nápoles. “A Review on the Long Short-Term Memory Model”. In: *Artificial Intelligence Review* 53.8 (Dec. 2020), pp. 5929–5955.
- [30] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning Internal Representations by Error Propagation”. In: ().
- [31] Pascal Vincent et al. “Extracting and Composing Robust Features with Denoising Autoencoders”. In: *Proceedings of the 25th International Conference on Machine Learning - ICML '08*. Helsinki, Finland: ACM Press, 2008, pp. 1096–1103.
- [32] Klaus Dorfmueller-Ulhaas. “Robust optical user motion tracking using a kalman filter”. In: (2007).
- [33] Ranch YQ Lai, Pong C Yuen, and Kelvin KW Lee. “Motion Capture Data Completion and Denoising by Singular Value Thresholding.” In: *Eurographics (short papers)*. 2011, pp. 45–48.

- [34] *CMU motion cap database*. Accessed: 2025-09-09. URL: <https://mocap.cs.cmu.edu/>.
- [35] Michael Burke and Joan Lasenby. “Estimating missing marker positions using low dimensional Kalman smoothing”. In: *Journal of biomechanics* 49.9 (2016), pp. 1854–1858.
- [36] Liqi Feng et al. “A Comparative Review of Graph Convolutional Networks for Human Skeleton-Based Action Recognition”. In: *Artificial Intelligence Review* 55.5 (June 2022), pp. 4275–4305.
- [37] Thomas N. Kipf and Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. Comment: Published as a conference paper at ICLR 2017. Feb. 2017. arXiv: 1609.02907 [cs].
- [38] Sijie Yan, Yuanjun Xiong, and Dahua Lin. *Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition*. Comment: Accepted by AAAI 2018. Jan. 2018. arXiv: 1801.07455 [cs].
- [39] Dario Pavlo et al. “3D Human Pose Estimation in Video With Temporal Convolutions and Semi-Supervised Training”. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Long Beach, CA, USA: IEEE, June 2019, pp. 7745–7754.
- [40] Qitao Zhao et al. “PoseFormerV2: Exploring Frequency Domain for Efficient and Robust 3D Human Pose Estimation”. In: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Vancouver, BC, Canada: IEEE, June 2023, pp. 8877–8886.
- [41] Yujun Cai et al. “Exploiting Spatial-Temporal Relationships for 3D Pose Estimation via Graph Convolutional Networks”. In: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. Seoul, Korea (South): IEEE, Oct. 2019, pp. 2272–2281.
- [42] Lei Shi et al. “Two-Stream Adaptive Graph Convolutional Networks for Skeleton-Based Action Recognition”. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Long Beach, CA, USA: IEEE, June 2019, pp. 12018–12027.
- [43] Ziyu Liu et al. *Disentangling and Unifying Graph Convolutions for Skeleton-Based Action Recognition*. Comment: CVPR 2020. May 2020. arXiv: 2003.14111 [cs].
- [44] Yi-Fan Song et al. “Stronger, Faster and More Explainable: A Graph Convolutional Baseline for Skeleton-based Action Recognition”. In: *Proceedings of the 28th ACM International Conference on Multimedia*. Seattle WA USA: ACM, Oct. 2020, pp. 1625–1633.
- [45] Wentian Xin et al. “Transformer for Skeleton-based Action Recognition: A Review of Recent Advances”. In: *Neurocomputing* 537 (June 2023), pp. 164–186.
- [46] Ashish Vaswani et al. “Attention Is All You Need”. In: ().
- [47] Kaiming He et al. “Masked Autoencoders Are Scalable Vision Learners”. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, June 2022, pp. 15979–15988.
- [48] Helei Qiu et al. *Spatio-Temporal Tuples Transformer for Skeleton-Based Action Recognition*. Comment: 14 pages, 5 figures. Jan. 2022. arXiv: 2201.02849 [cs].
- [49] *PyTorch Framework*. Accessed: 2025-09-09. URL: <https://pytorch.org/>.
- [50] Diederik P Kingma. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [51] *ReduceLROnPlateau*. Accessed: 2025-09-09. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html#reducelronplateau.
- [52] *BatchNorm*. Accessed: 2025-09-09. URL: <https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html#torch.nn.BatchNorm2d>.

Bibliography

- [53] *anglespan*. Accessed: 2025-09-09. URL: https://support.cognex.com/docs/cv1_900/web/EN/cv1_users_guide/Content/Topics/Angle_Span.htm?TocPath=Shapes%7CShape%20Geometries%7CPolygons%20and%20Wireframes%7Ccc2Wireframe%7CSegments%7C_____1.
- [54] Chang Ko and Daesoo Kim. “Reconstruction of Postures Using Partial Body Information Through a Self-supervised Transformer in Mice”. In: ().